WİNDOWS NEVER FORGETS

A Field Guide to What Your PC Remembers, and How Forensic Investigators Read It Back

Emil Veliyev

github.com/emillvl

WINDOWS INTERNALS · TELEMETRY · DIGITAL FORENSICS

Windows Never Forgets

A Field Guide to What Your PC Remembers, and How Forensic Investigators Read It Back

Emil Veliyev

github.com/emillvl

Windows Never Forgets

A Field Guide to What Your PC Remembers, and How Forensic Investigators Read It Back

Written and compiled by Emil Veliyev

github.com/emillvl

First compiled edition.

This book is a personal technical reference on Windows telemetry, digital forensics, and internals, covering Windows 10, Windows 11, and Windows Server 2016–2022.

Preface

Every modern version of Windows keeps a diary. Nobody sits down and writes it on purpose. It accumulates on its own, a byproduct of hundreds of small subsystems doing their jobs: a service that wants to know if your last update installed cleanly, a shell feature that wants to show you your recent files, a diagnostics pipeline that wants to tell Microsoft which drivers are crashing. None of these were designed as surveillance tools. Almost all of them were designed to make the operating system faster, more reliable, or more convenient. But the side effect, added up across the whole system, is a machine that remembers an enormous amount about what you did, when you did it, and often what you were looking at while you did it.

This book is a tour of that diary. It covers three things at once, because on Windows they turn out to be the same subject looked at from different angles. The first is telemetry: the deliberate data collection Microsoft built into the operating system, the services that run it, and the servers it phones home to. The second is digital forensics: the incidental record-keeping that investigators rely on to reconstruct what happened on a machine, whether that machine belongs to a crime victim, an insider threat, or a curious owner who wants to know what their own computer has been doing. The third is persistence and internals: the kernel-level plumbing, the registry structures, and the file formats that make all of the above possible, and that attackers exploit for their own purposes when they want a foothold that survives a reboot.

The book does not assume you already know this material. It assumes you can read English and are comfortable with the

general idea of files, folders, and settings. Everywhere a piece of jargon shows up for the first time, it gets explained in plain language before the book leans on it. A short list of recurring concepts follows this preface, and you can skip straight past it if terms like "registry hive" or "FILETIME" are already familiar.

At the same time, this book does not round anything off. Every registry key, every file path, every event ID, every field inside every binary structure that shows up in the source research behind this book shows up here too. The goal is not a simplified summary that leaves the specifics out. It's the opposite: all the specifics, explained well enough that you don't need a forensics certification to follow along. If you already work in this field, the exact paths and structures are all still here for you to use as reference. If you don't, the surrounding explanation should carry you through without needing to look anything up elsewhere.

A note on scope and currency: the material draws on Windows 10 (from roughly the 20H2 update through 22H2) and Windows 11 (from 21H2 through 24H2, including the Recall feature introduced for Copilot+ PCs), plus Windows Server 2016 through 2022 where server-specific behavior differs. Registry paths, file formats, and service names shift a little between builds. Where the underlying research flagged a detail as version-dependent, this book says so.

A few terms worth knowing before you start

The registry. Windows stores most of its configuration, and a surprising amount of its history, in a hierarchical database called the registry. Think of it as a filing cabinet of settings, organized into nested folders called keys, each of which can hold named values. The two cabinets you'll see most often in this book are

`HKEY_LOCAL_MACHINE` (shortened to `HKLM`, holding settings for the whole machine) and `HKEY_CURRENT_USER` (`HKCU`, holding settings for whichever user is logged in). On disk, these cabinets are stored in files called hives: the machine-wide hives live under `%SystemRoot%\System32\config\`, and each user's personal hive is a file called `NTUSER.DAT` inside their profile folder, with a second hive, `UsrClass.dat`, under `%LocalAppData%\Microsoft\Windows`.

FILETIME. Windows likes to record time as a single 64-bit number: the count of 100-nanosecond intervals since midnight on January 1, 1601. That odd starting point comes from an old Microsoft convention, not from anything meaningful happening in 1601. You'll see this format constantly, because Windows uses it for almost every timestamp buried inside a registry value or binary artifact. Chapter 42 collects every conversion formula this book uses in one place.

ETW (Event Tracing for Windows). A logging framework built into the Windows kernel itself. Software components called providers emit structured events (a process started, a file was written, a registry key was queried); other components called consumers subscribe to those events and do something with them, whether that's writing them to a file, displaying them in a monitoring tool, or feeding them into Microsoft's telemetry pipeline. ETW runs constantly, mostly invisibly, and it underpins a large fraction of what this book describes. Part seven is dedicated to it.

DFIR. Short for Digital Forensics and Incident Response: the discipline of reconstructing what happened on a computer after the fact, usually because something went wrong. A forensic investigator doesn't get to ask the computer what happened. They have to infer it from artifacts like the ones in this book: a timestamp here, a cached file path there, a registry value that

only updates under specific conditions. Every chapter in this book eventually turns to the question a DFIR investigator would ask: what can I actually prove from this artifact, and what are its limits?

Telemetry. Microsoft's term for the data Windows collects about itself and sends back to Microsoft, covering everything from crash reports to feature usage statistics to (depending on your settings) a fair amount of behavioral data. It is distinct from forensic artifacts in one important way: telemetry is designed to leave the machine, while forensic artifacts are usually just local byproducts that happen to stay put. In practice the two categories overlap constantly, since the same local file that feeds Microsoft's telemetry pipeline is often the same file an investigator would pull off a hard drive.

With that vocabulary in place, the rest of the book moves in roughly the order you'd want to investigate a machine yourself: first the background services running, then the traces of what got executed, then what got clicked and opened, then the file system's own record-keeping, then the deeper kernel and tracing machinery underneath it all, and finally the network, cloud, and credential systems that tie a single PC to a Microsoft account and a wider enterprise. It closes with a set of reference chapters: tools, timestamp math, what anti-forensic techniques defeat and what they don't, and the registry keys that actually turn each piece of collection off.

Contents

PART ONE: THE WATCHERS 11

1. The telemetry and diagnostics machinery 12
2. Identity, location, and the cloud tether 16
3. Network, security, and shell services 20

PART TWO: WHAT YOU RAN 24

4. Prefetch, ShimCache, and Amcache: three ways of proving a program existed 25
5. SRUM: an hour-by-hour diary of resource use 29
6. BAM, DAM, and the last-run timestamp 31
7. Windows Recall: when the OS takes its own screenshots 33

PART THREE: WHAT YOU CLICKED 35

8. UserAssist and the ROT13 ledger 36
9. The MRU universe: recent documents, dialogs, and typed paths 38
10. Shellbags: folders that remember you visited 42
11. LNK files and jump lists: shortcuts and taskbar menus that testify 44

PART FOUR: THE FILE SYSTEM ITSELF 49

12. The master file table and the two kinds of timestamps 50
13. $LogFile, $UsnJrnl, and $Recycle.Bin 53

PART FIVE: WHAT THE SYSTEM SAW 56

14. Windows event logs, channel by channel 57

15. When Windows crashes: WER, minidumps, and kernel dumps 60

16. Problem Steps Recorder, boot and shutdown events, hibernation, and the page file 64

PART SIX: THE SHELL REMEMBERS 68

17. Thumbnails, icons, and the Windows search index 69

18. Windows Timeline and the activities cache 72

19. Microsoft Edge's local footprint 75

20. Notifications, Cortana, Internet Explorer's legacy cache, maps, clipboard, and sticky notes 78

PART SEVEN: UNDER THE HOOD 81

21. How the kernel watches every process, thread, and image load 82

22. Registry callbacks, object callbacks, and minifilters 85

23. ETW, properly explained 88

24. The autologgers that start before you log in, and the providers they subscribe to 90

PART EIGHT: THE SCHEDULER'S SECRETS 95

25. The scheduled tasks Windows runs without asking 96

26. How the Task Scheduler keeps score 100

PART NINE: THE NETWORK AND THE CLOUD 103

27. The telemetry pipeline's journey to Microsoft 104

28. Windows Update, Delivery Optimization, and identity sync 106

29. OneDrive and the Connected Devices Platform 108

30. Push notifications, SmartScreen, MAPS, and cloud clipboard 112

31. Wi-Fi profiles, Credential Manager, and token stores 115

32. Xbox, Edge sync, the Microsoft Store, and the enterprise endpoint list 118

PART TEN: PERSISTENCE AND THE EARLY BOOT 122

33. BITS, BootExecute, MountedDevices, and the ordinary autostart locations 123

34. WMI and its persistence tricks 128

PART ELEVEN: SCRIPTS, AUDITS, AND WHAT GETS LOGGED 130

35. PowerShell is always watching itself 131

36. The DNS cache, security auditing, and AppLocker 134

PART TWELVE: CREDENTIALS, IDENTITY, AND DEFENDER 138

37. Windows Hello, DPAPI, and the credential vaults 139

38. Defender's detection history, certificate stores, and the firewall log 142

PART THIRTEEN: THE LAST MILE 145

39. Installer footprints, the services registry, and Group Policy artifacts 146

40. The EVTX format itself 149

PART FOURTEEN: MAKING SENSE OF IT ALL **151**

41. Tools of the trade 152

42. Making sense of timestamps 154

43. Anti-forensics: what defeats each artifact, and what survives 156

44. The privacy control registry: turning it off 159

PART ONE

the watchers

CHAPTER 1

The telemetry and diagnostics machinery

Open Task Manager on any Windows 10 or 11 machine and you'll find dozens of `svchost.exe` processes, each one hosting one or more background services. Most people never look past that. But a cluster of those hidden services exists purely to watch the operating system run, catch it when it breaks, and report both facts back to Microsoft. This chapter walks through that cluster: the core telemetry pipeline, the crash-reporting system that catches it when something goes wrong, and the services that specifically watch how you use your applications.

The core telemetry and diagnostics pipeline

At the center of it all sits a service called **Connected User Experiences and Telemetry**, known internally as `DiagTrack`. It runs inside a shared `svchost.exe` process using the switch `-k utcsvc -p`, backed by the file `%SystemRoot% \System32\diagtrack.dll`, and its configuration lives at `HKLM\SYSTEM\CurrentControlSet\Services\DiagTrack`. DiagTrack is the closest thing Windows has to a single telemetry funnel: it collects your OS version and hardware profile, which apps you've installed and uninstalled, search queries, Edge browsing patterns, samples of your handwriting and typing (if you've enabled those features), voice clips where applicable, crash summaries, feature usage flags, and a fingerprint of what your device is capable of. Whether any of that actually leaves the machine depends on your telemetry level setting, and the minimum level depends on your

Windows edition: Home/Pro require Basic; Enterprise, Education, and Server also support diagnostic data off.

DiagTrack doesn't work alone. A service called dmwappushservice, the WAP Push Message Routing Service, routes OMA-DM push messages used for mobile device management enrollment, and in the process it can silently bootstrap DiagTrack enrollment itself. The Diagnostic Policy Service (`DPS`) makes automated decisions about troubleshooting, logging its results to `%SystemRoot%\System32\LogFiles\WMI\`. Two more services, `WdiServiceHost` and `WdiSystemHost`, host Windows Diagnostic Infrastructure plugins: one runs as Local Service and reacts to things like resource exhaustion or dropped network connections, the other runs at the system level and handles infrastructure-wide diagnostics. The Windows Insider Service (`wisvc`) manages Insider ring enrollment and pulls feature flags from `settings-win.data.microsoft.com`, storing your flight token locally. And the Diagnostic Execution Service (`diagsvc`) can execute on-demand diagnostic scripts pushed down through the telemetry pipeline itself, saving its results to `%ProgramData%\Microsoft\Diagnosis\`.

These services have different dependencies; the diagnostic-data level does not govern all of them. "Enhanced" is a legacy level, unavailable on Windows 11 and Server 2022. What's worth noticing is the shape of the system: it isn't one monolithic spy service, it's a small constellation of purpose-built agents, each with its own registry key and its own narrow job, that together add up to continuous low-level monitoring of how the machine is behaving.

When things break: Windows Error Reporting

The moment an application crashes, a second machinery takes over: Windows Error Reporting, or WER. The `WerSvc` service

(hosted from `%SystemRoot%\System32\WerSvc.dll`) manages a queue of crash reports at `%ProgramData% \Microsoft\Windows\WER\ReportQueue\`, moving them to `%ProgramData%\Microsoft\Windows\WER\ReportArchive\` once they've been handled. Two companion executables, `WerFault.exe` and `WerFaultSecure.exe`, support crash reporting. `WerFault.exe` handles ordinary user-mode faults; . wer metadata and attachments belong to `WER` report folders. Separately, opt-in `LocalDumps` can write user-mode dumps to `%LocalAppData%\CrashDumps\.` Uploads depend on settings and consent. Kernel bugcheck dumps use a separate mechanism, with locations governed by CrashControl.

You can inspect and adjust how aggressively this system behaves through a small cluster of registry values under `HKLM\SOFTWARE\Microsoft\Windows\Windows Error Reporting\`. `AutoApproveOSDumps` controls whether kernel dumps get submitted automatically. `DontSendAdditionalData` and `Disabled` do roughly what they say. `LoggingDisabled` turns off WER's local logging entirely. A subkey called `LocalDumps` governs where dumps get written (`DumpFolder`), what kind gets kept (`DumpType`, where 0 means a custom dump, 1 a minidump, and 2 a full memory dump), and how many are retained (`DumpCount`). An older, largely vestigial key, `HKLM\SOFTWARE\Microsoft\PCHealth\ErrorReporting\`, still exists from earlier Windows versions.

The practical effect is that almost any crash on a Windows machine, whether it's Notepad hanging or a kernel driver bugchecking the whole system, leaves a structured, timestamped trail sitting in a predictable folder, whether or not it ever gets uploaded anywhere.

Watching how you use your programs

A third cluster of services exists to monitor application behavior and compatibility rather than catastrophic failure. The Program Compatibility Assistant Service (`PcaSvc`, from `%SystemRoot% \System32\pcasvc.dll`) watches every process launch for compatibility problems, records launch data on Windows 11 22H2 in `%SystemRoot%\AppCompat\pca\PcaAppLaunchDic.txt`, and feeds two other artifacts covered later in this book, AppCompatCache and Amcache. `SysMain`, the modern name for what used to be called SuperFetch, tracks how often you launch applications and how you use RAM during a session, writing its findings as `.db` files into `%SystemRoot% \Prefetch\`. The Application Information service (`Appinfo`) handles UAC elevation requests. When process-creation auditing is enabled, Security event ID 4688 records new processes, including their process ID, creator process ID, image path, and account context. A now-deprecated service, `DcpSvc` (the Diagnostic Data Collector), used to gather app diagnostics and forward them into the DiagTrack pipeline. And the Windows Performance Recorder runtime, `wpr.exe`, can be triggered by scheduled tasks to capture full ETW traces into `.etl` files under `%ProgramData% \Microsoft\Diagnosis\ETLLogs\`.

Individually, none of these services is dramatic. Together, they mean that program launches, UAC prompts, RAM usage patterns, and compatibility quirks are all being logged somewhere on disk as a matter of routine, long before anyone goes looking for evidence of anything in particular.

CHAPTER 2

Identity, location, and the cloud tether

The second cluster of background services has a different focus: not how the machine is running, but who is using it, where they are, and how their device connects back to a Microsoft account or an organization's device-management system.

Managed machines

If your PC belongs to a company or school, it's likely enrolled in mobile device management (MDM), and a handful of services exist purely to support that. The Device Management Enrollment Service (`DmEnrollmentSvc`) tracks enrollment state and writes policy configuration through OMA-URI, storing account association data in `%ProgramData%\Microsoft\DMClient\`. The Enterprise App Management Service (`EntAppSvc`) logs install and uninstall events for enterprise-deployed apps along with sideloading policy. A WMI-based component sometimes called the DM Client acts as a bridge to the full device configuration state, which lands in `%ProgramData%\Microsoft\DMClient\{accountGUID}\`, with its account list stored at `HKLM\SOFTWARE\Microsoft\Provisioning\OMADM\Accounts\`. And a component tied to Windows Autopilot, hosted from `CloudExperienceHostBroker.dll`, tracks the device's join state and computes a hardware hash used to match it against Autopilot records at `HKLM\SOFTWARE\Microsoft\CloudDomainJoin\`.

The data these services generate collects in a predictable set of locations: `%ProgramData%\Microsoft\DMClient\`, a JSON file called

`DeviceApplicability.json` inside a GUID-named subfolder there, `%SystemRoot%\System32\config\systemprofile\AppData\Local\MDMDiagnostics\`, and a dedicated Windows Event Log channel, `Microsoft-Windows-DeviceManagement-Enterprise-Diagnostics-Provider`.

Where you are, who you are

A separate group of services handles location and biometric data. The Geolocation Service (`lfsvc`) derives your latitude and longitude from GPS, Wi-Fi, or cell tower data, and stores per-app consent decisions under a long GUID-named key inside `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\DeviceAccess\Global\`. The Sensor Data Service aggregates readings from motion, light, and proximity sensors. The Windows Biometric Service (`WbioSrvc`) stores fingerprint and face templates in `%SystemRoot% \System32\WinBioDatabase\` and logs every match or non-match attempt along with enrollment events. Two more services, `SensrSvc` and SensorService, support automatic brightness adjustments and sensor functions including orientation and history, respectively.

None of this is inherently alarming; it's what makes "find my device" style features and adaptive brightness work. But each of these services keeps its own local record, and each one is a potential source of evidence about a device's physical location and its owner's biometric enrollment history.

The cloud tether

The largest and most interconnected group of services in this chapter exists to keep your PC synchronized with a Microsoft

account, and by extension with your other devices. The Connected Devices Platform (`CDPSvc`, plus a per-user counterpart `CDPUserSvc`) supports cross-device features, including clipboard and Nearby sharing; older Timeline sync used `activityfeed.microsoft.com` and caching locally in `%LocalAppData%\ConnectedDevicesPlatform\`. `OneSyncSvc` (hosted from `APHostService.dll`) is the sync framework behind mail, calendar, and contacts for UWP apps. The Clipboard User Service (`cbdhsvc`) handles cloud clipboard sync through your Microsoft account, keeping local history at `%LocalAppData%\Microsoft\Windows\Clipboard\`. The Microsoft Account Sign-in Assistant (`wlidsvc`) refreshes your account tokens against `login.live.com` and caches them at `%LocalAppData% \Microsoft\Windows\AccountState\`.

Gaming has its own pair of services here too: `XblAuthManager` handles Xbox Live authentication and links your device to your gamertag, while `XblGameSave` uploads game save data to Xbox cloud storage, syncing per-game state. Settings Sync, run through the hosted process `SettingSyncHost.exe`, pushes your theme, saved passwords, language preferences, and (if you opt in) Wi-Fi credentials and browser settings up to your Microsoft account cloud, tracked under `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\SettingSync\Groups\`. And two notification services, `WpnService` and its per-user counterpart `WpnUserService`, register your device's push notification channel with `client.wns.windows.com` and store per-app notification history in `%LocalAppData%\Microsoft\Windows\Notifications\wpndatabase.db`, a database this book returns to in more detail in part six.

Taken together, this chapter's services are the reason a Windows PC doesn't feel like an isolated box: your theme, your files, your clipboard, and your recent activity all follow you across

devices automatically, because a dozen small services are constantly reading and writing that state in the background.

CHAPTER 3

Network, security, and shell services

The last cluster of background services in this part spans networking infrastructure, Defender's own telemetry, search and shell functionality, a Server-only auditing feature, and a long tail of smaller agents that don't fit neatly anywhere else.

The network layer's watchers

Several core networking services do double duty as data sources. Network Location Awareness (`NlaSvc`) tracks network context; NCSI tests internet reachability, and writes the SSID, gateway MAC address, and domain membership it observes into the registry. The DNS Client (`Dnscache`) holds an in-memory cache of resolved names and negative lookups. DHCP Client (`Dhcp`) records your assigned IP, gateway, DNS servers, and lease times under `HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces\{GUID}`. WLAN AutoConfig (`WlanSvc`) stores every Wi-Fi profile you've ever connected to, passwords included, in `%ProgramData% \Microsoft\Wlansvc\Profiles\Interfaces\`, with connection history captured through ETW; WWAN AutoConfig does the cellular equivalent, tracking carrier, IMSI, IMEI, and data usage.

A few of these services are less well known but forensically significant. The Distributed Link Tracking Client (`TrkWks`) tracks NTFS files as they move across volumes using object IDs, a mechanism that resurfaces later in this book when we look at LNK files and the `$OBJECT_ID` NTFS attribute. IP Helper (`iphlpsvc`) tracks tunnel adapter state for IPv6 transition technologies like

6to4, Teredo, and IP-HTTPS, and Teredo specifically can reveal a device's presence behind NAT. When debug logging is enabled, Netlogon logs authentication and secure-channel activity to `%SystemRoot%\debug\netlogon.log.` BITS, the Background Intelligent Transfer Service, queues asynchronous downloads in `%ProgramData% \Microsoft\Network\Downloader\` and gets used by Windows Update, Defender, and the telemetry pipeline itself; part ten covers it in depth, including how attackers repurpose it for persistence. And WebClient, the WebDAV client, transfers files and metadata over HTTP or HTTPS while Explorer keeps mapped paths under its "Map Network Drive `MRU`" key.

Defender's own telemetry services

Windows Defender is itself a telemetry source as much as a security tool. The main antivirus engine (`WinDefend`, running as `MsMpEng.exe`) computes SHA-256 hashes of files for cloud lookups against Microsoft's protection service and reports detection and behavioral events to a family of endpoints under `*.wdcp.microsoft.com`. `WdNisSvc` performs network inspection, correlating traffic events with the process responsible. For organizations running Microsoft Defender for Endpoint, the `Sense` service streams a much richer feed: selected process, network, file, registry, logon, and memory-related events, sent to Microsoft's Azure-hosted analytics pipeline. Two smaller services round this group out: `wscsvc`, the Security Center, tracks antivirus and firewall registration state for the Action Center, and `mpssvc`, the Defender Firewall service, logs blocked connections as Security event ID 5157.

Search, shell, and the rest

Windows Search (`WSearch`, running as `SearchIndexer.exe`) builds a full-text index of your documents, mail if Outlook is configured, file metadata, and even PDF content, using `Windows.edb` or `Windows.db`, described fully in part six. Shell Hardware Detection responds to AutoPlay events and records removable media serial numbers and volume GUIDs. The Downloaded Maps Manager tracks your offline map downloads and your last region of interest. Themes syncs your desktop customization choices, which then flow out through Settings Sync described in the previous chapter.

On Windows Server editions only, a feature called User Access Logging (`UALSVC`) keeps up to three years of records, in Microsoft Jet/ESENT database format, of client activity reported by roles and products that support UAL on that server, including client IP addresses, usernames, and access timestamps. The databases live at `%SystemRoot%\System32\LogFiles\Sum\`, split into a `Current.mdb` for daily activity, GUID-named databases for the current and prior years, and a `SystemIdentity.mdb` holding hardware identifiers and the last five characters of the product key.

Finally, a long tail of smaller services fills in specific gaps: Group Policy application (`gpsvc`) writes resultant policy data and GPO history; `ProfSvc` logs user profile load and unload events; `StorSvc` handles Storage Sense logs and per-volume write statistics; `StateRepository` maintains a database of every installed and uninstalled UWP app with timestamps; `TokenBroker` caches OAuth2 tokens for UWP apps; `fhsvc` runs File History backups and catalogs them in an ESE database on the backup drive; `TimeBrokerSvc` and `SystemEventsBroker` handle background task scheduling and trigger events; a pair of per-user services, `PimIndexMaintenanceSvc` and `UserDataSvc`, maintain contact and

calendar indexes backed by an ESE database named store.vol, used by Windows Mail; `AppReadiness` timestamps app installations per new user account; `RetailDemoSvc` orchestrates retail demo mode; and `RmSvc` tracks wireless radio state, recording every time you flip airplane mode on or off.

None of these individually would justify a whole book. But this is the point of part one: before you can understand what a forensic artifact means, or what a piece of telemetry contains, it helps to know which of these dozens of quiet services put it there, and why.

PART TWO

what you ran

CHAPTER 4

Prefetch, ShimCache, and Amcache: three ways of proving a program existed

If you ask three different parts of Windows whether a given program ran, you can get three different kinds of answer, because three unrelated subsystems each keep their own partial record for their own reasons. None of them was built as a forensic tool. All three get used as one anyway.

Prefetch

Prefetch exists to make programs launch faster. Every time you run an executable, Windows' memory manager watches which files and directories get touched in the first several seconds and writes a summary of that activity to a file in `%SystemRoot%\Prefetch\`, named after the executable plus a hash of its launch path, so `notepad.exe` might produce something like `NOTEPAD.EXE-3AF16A5C.pf`. Whether this happens at all, and for what, is controlled by a registry value at `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management\PrefetchParameters\EnablePrefetcher`: 0 disables it, 1 limits it to applications, 2 limits it to boot-time prefetching, and 3 (the default on workstations) enables both. Server editions typically ship with it disabled.

Windows 10 introduced compressed v30 Prefetch files, built with the Xpress Huffman variant of LZXPRESS compression. Once decompressed, the file starts with a version and a four-byte

signature reading `SCCA`, reserved bytes, the decompressed file's size, and then sixty bytes holding the executable's name in UTF-16LE. From there the file records a hash of the launch path, a run count, and, on Windows 8 and later, the timestamps of the eight most recent executions (Windows 7 kept only one). It goes on to list every volume referenced during those runs, the DLLs and other resources loaded in the first ten seconds of execution, and every directory the process touched. That DLL list turns out to be one of the file's most useful forensic properties: it's a snapshot of what got loaded alongside the program, which makes injected or unexpected modules stand out. The limit is 128 files on Windows 7, normally 1,024 on Windows 8 and later, rotating out the oldest when that limit is reached, and the standard tool for parsing them into readable CSV or JSON is Eric Zimmermann's `PECmd.exe`.

ShimCache, also called AppCompatCache

Where Prefetch is a file, ShimCache lives entirely in the registry, at `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache\AppCompatCache` (worth checking the `ControlSet001` and `ControlSet002` variants too), as a single binary blob holding up to 1,024 entries. It exists to support the Application Compatibility subsystem, which needs to know which executables it has already evaluated for compatibility shims. Each entry records a full NT-style device path and a timestamp, but that timestamp is not when the program ran: it's the file's last-modified time as recorded in its `$STANDARD_INFORMATION` attribute at the moment the cache was updated. On Windows 7 and 8, entries carried an explicit "executed" flag; on Windows 10 and 11 that flag was removed entirely. This is the single most important thing to know about ShimCache: on a modern Windows install, a file's presence in the cache proves only that the file existed and was seen by the

filesystem, not that it ran. The cache is also only written to disk on a clean shutdown, so its contents lag behind whatever's currently loaded in memory. The binary blob starts with a signature reading `10ts` on Windows 10, and the standard parsing tool is `AppCompatCacheParser.exe`, again from Zimmermann's suite.

Amcache

The third artifact, Amcache, is a genuine registry hive of its own, sitting at `%SystemRoot%\AppCompat\Programs\Amcache.hve`, populated by the Program Compatibility Assistant service working alongside the broader AppInfo and app-compatibility stack. Unlike ShimCache, it isn't loaded into the live registry the way `HKLM` and `HKCU` are; it has to be read offline or through a volume shadow copy. Its structure is organized under a root key with several major branches. `InventoryApplicationFile` holds one subkey per executable, named after a hash of its file path, and each of those subkeys records a `FileId` value whose last forty hex characters are a SHA-1 hash of up to the first 31,457,280 bytes (30 MiB) of the file (useful for threat-intelligence lookups even after the file itself is deleted), along with the file's full lowercase path, its PE linker timestamp, size, product name and version, publisher, and a foreign key linking it to an installed-application record. On Windows 11 22H2 and later, some builds have been observed recording SHA-256 hashes instead. `InventoryApplication` catalogs installed software with install and uninstall dates, publisher, install source, and uninstall string. `InventoryDriverBinary` tracks driver files with their checksum, signing status, and kernel-mode flag. `InventoryDevicePnp` inventories connected hardware devices.

The `LastWriteTime` of an `InventoryApplicationFile` key records an inventory update, which

approximates when that entry was last touched, not when the executable ran. Because the hive is locked while Windows is running, collecting it requires either a volume shadow copy or an offline parse, typically with `AmcacheParser.exe`.

Put the three together and a pattern emerges that recurs throughout this book: no single Windows artifact tells the whole story on its own. Prefetch gives you up to eight precise execution timestamps but within a finite cache, normally up to 1,024 files on Windows 8 and later. ShimCache gives you a long list of files the system has seen but, on modern Windows, no proof any of them executed. Amcache gives you rich metadata, including a hash you can use even after the file is gone, but only an approximate sense of timing. Investigators cross-reference all three specifically because each one's weaknesses are covered by one of the others' strengths.

CHAPTER 5

SRUM: an hour-by-hour diary of resource use

The System Resource Usage Monitor, SRUM, takes a different approach entirely: instead of logging individual events, it keeps an hourly ledger of how much of the machine's resources every application consumed. It's implemented through `srumsvc.dll`, running inside a shared `svchost.exe` process, and its storage engine is ESENT (also called the Extensible Storage Engine, or JET Blue), the same database technology Windows uses for a handful of other artifacts in this book. The database itself lives at `%SystemRoot%\System32\sru\SRUDB.dat`, with a companion transaction log at `SRU.log` capturing pending writes before they're flushed. SRUM flushes to disk roughly once an hour and again at shutdown, and each of its data-collecting extensions is registered under `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\SRUM\Extensions\`, one GUID-named key per extension, each recording when that extension started collecting and its collection mode.

Inside the database, a global lookup table called `SruDbIdMapTable` maps small integers to the actual strings they represent, whether that's a SID, an executable path, or an Application User Model ID for a packaged app; every other table in the database refers back to entries here rather than repeating long strings. A `SruDbCheckpointTable` tracks internal checkpointing.

The centerpiece is the Application Resource Usage extension, identified by the GUID `{D10CA2FE-6FCF-4F6D-848E-B2E99266FA89}`. One row in this table represents one application, for one user,

during one hour-long window, and the columns are remarkably granular: separate CPU cycle counts for foreground and background execution, a `FaceTime` column measuring actual foreground wall-clock duration, context-switch counts, and separate read and write byte counts, I/O operation counts, and flush counts for foreground versus background activity. A handful of companion extensions round out the picture: a Push Notification Provider table records payload sizes and network types per app per hour; a Network Data Usage table (GUID `{973F5D5C-1D90-4944-BE8E-24B94231A174}`) tracks bytes sent and received per network interface; a Network Connectivity table records how long each interface stayed connected and under which Wi-Fi or Ethernet profile name; and a pair of Energy Usage tables, one short-term and one long-term aggregate, track battery state transitions, designed and full-charge capacity, time spent on AC versus battery power, and energy consumed while active.

`SRUDB.dat` is usually locked while Windows runs; live collection can use a shadow copy, raw `NTFS` access, or a copy after stopping the Diagnostic Policy Service (`DPS`). The `AppId` values throughout the database resolve, through that central ID map table, to either a classic Win32 path or an AUMID string like `Microsoft.MicrosoftEdge_8wekyb3d8bbwe! MicrosoftEdge` for packaged apps. Many `SRUM` records combine activity over an hour, so their timestamps alone do not show exactly when a program ran; that's what Prefetch and BAM are for. Data typically survives for thirty to sixty days depending on how large the database has grown. The standard offline parsing tools are the open-source `srum-dump` and Zimmermann's `SrumECmd.exe`. Forensically, the `FaceTime` column is often the single most useful field in the whole database, since it's the cleanest way to distinguish software the user was actually looking at from software merely running in the background.

CHAPTER 6

BAM, DAM, and the last-run timestamp

Background Activity Moderator (BAM) and its counterpart Desktop Activity Moderator (DAM) take yet another approach: instead of a database, they're kernel-mode drivers, `bam.sys` among them, registered to observe process execution through the Power Manager, and they write directly into the SYSTEM registry hive rather than maintaining any file of their own. That design choice matters, because it means BAM and DAM entries are resilient to a lot of the log-clearing techniques that work against file-based artifacts.

Execution timestamps land under `HKLM\SYSTEM\CurrentControlSet\Services\bam\State\UserSettings\{UserSID}\`, with related dam keys on supported builds, and each subkey is scoped to a particular user's SID. Two additional values, `Version` and `SequenceNumber`, live under `bam\State\` and track the driver's own versioning. Inside a user's subkey, a recorded binary appears as its own registry value, named with its full NT device-form path (something like `\Device\HarddiskVolume3\Windows\System32\cmd.exe`), holding a 24-byte binary blob. The first eight bytes are a FILETIME recording the last time that binary ran, in UTC; the next four bytes are usually zero padding; the following four are a sequence or entry counter; the final eight bytes are typically unused.

BAM's most useful trait, and its clearest limitation, is that it records only the single most recent execution timestamp per path, never a running count. It has no memory of how many times

something ran, only of the last time. But that one timestamp is unusually durable: entries can survive reboots, but researched Windows 10 builds purge entries older than seven days during boot, and because BAM writes are keyed to a specific user SID, this becomes especially useful in lateral-movement investigations, where an attacker moving between accounts leaves BAM entries scattered under each victim account's own subkey rather than the attacker's. On a live, running system, the `bam\State` key sits inside the SYSTEM hive, which is locked by the kernel, so the key can still be queried live with sufficient permissions; offline hive copies, shadow copies, and memory acquisitions offer other ways to read it. A secondary, coarser timestamp is available too: the `LastWriteTime` of the user's SID subkey itself, which marks the last time BAM updated any entry for that user, useful when someone has tampered with individual value data but not touched the key's own metadata. On Windows 11, the picture gets slightly more complex, since BAM also tracks package-relative paths for MSIX-packaged apps, which show up with paths under `Program Files\WindowsApps` rather than a traditional executable location.

CHAPTER 7

Windows Recall: when the OS takes its own screenshots

The newest artifact in this part is also the most sweeping. Windows Recall, introduced for Copilot+ PCs running Windows 11's 24H2 update, when the user opts in, periodically captures snapshots of eligible screen content, runs OCR and semantic analysis over it, and stores the result in a searchable local database, effectively building a visual timeline of everything you've looked at.

The feature runs as a user-mode process (`AIXHost.exe` or `ScreenshotManager.exe`, depending on build) backed by a runtime called CoreAIPlatform, whose primary binary is `%SystemRoot% \System32\CoreAIPlatform.dll`. Everything Recall captures lands under `%LocalAppData%\CoreAIPlatform.00\UKP\{UserGUID}\`, where the GUID is assigned to your account at Recall enrollment. Inside that folder sits a SQLite database, `ukg.db`, and encrypted image data under ImageStore. Keys are protected by TPM hardware, Windows Hello sign-in, and a VBS enclave, a protected area of memory. Paths and formats vary by build. Recall also logs to its own event channel, `Microsoft-Windows-WindowsAI/Operational`.

The database schema explains exactly what gets recorded. In builds examined by TotalRecall, `WindowCapture` records timestamps, titles, bounds, and an `ImageToken` column. `WindowCaptureTextIndex` indexes window titles and OCR text; related tables link captures to applications, websites, files, and topics. The tables vary by build, and this metadata can describe browsing, editing, viewing,

communicating, and other activities. The structures holding OCR text vary by build. The `App` table maps application identifiers to names, paths, and icon metadata.

That indexed OCR text is, by a wide margin, the most sensitive part of the whole system. It captures OCR'd text from whatever was visible on screen at capture time, which means it can include passwords typed into fields that weren't masked, personal information displayed in a document, or the contents of a private message, all indexed and searchable. The released design uses Windows Hello Enhanced Sign-in Security (ESS) and the enclave to protect access to the encrypted snapshots and index. An NTLM hash and ordinary `DPAPI` master-key files alone do not provide those Recall keys. Recall can be switched off through `HKCU\SOFTWARE\Policies\Microsoft\Windows\WindowsAI\DisableAIDataAnalysis=1`, and retention depends on configured duration and storage limits; three months is not a universal maximum. On a system where Recall is enabled and later compromised, the combination of `ukg.db` and its encrypted screenshots can reveal sensitive screen content from eligible snapshots kept during that window, which is exactly why the feature has drawn as much attention as it has since its introduction.

PART THREE

what you clicked

CHAPTER 8

UserAssist and the ROT13 ledger

UserAssist is the registry's record of GUI program launches specifically: anything started from the Start menu, Explorer, a desktop shortcut, or the Run dialog. It does not see programs launched from a command line, from PowerShell, or directly through the Windows API, which makes it a companion to Prefetch and BAM rather than a replacement for either.

It lives at `HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\`, organized into a small number of GUID-named subkeys, each holding a `Count\` key underneath it. The two that matter on a modern system are `{CEBFF5CD-ACE2-4F4F-9178-9926F41749EA}`, which tracks execution of ordinary application executables, and `{F4E57C4B-2036-45F0-A9AB-443BCFE33D9F}`, which tracks execution via `.lnk` shortcut files specifically. A handful of older, largely vestigial GUIDs, associated with Windows XP and the original Internet Explorer, still occasionally turn up on systems with a long upgrade history but rarely contain anything current.

Every value name under `Count\` (which is itself the full path of the program that ran) is obfuscated with ROT13, a simple substitution cipher that shifts every letter thirteen places through the alphabet, wrapping from Z back to A. It is not encryption in any meaningful sense, just a light scramble meant to stop a human from reading it at a glance: `HFRE\`Qrfxgbc`\`zlrkr.rkr decodes straightforwardly to `USER\`Desktop`\myexe.exe`. The `GUID` key names themselves are not ROT13-encoded. Numbers, backslashes,

colons, spaces, and braces all pass through unchanged, since ROT13 only touches letters.

Each decoded value holds a 72-byte binary blob recording execution statistics. The layout that's been experimentally confirmed for Windows 10 and 11 places a session ID at offset 0x00, a run count at 0x04, a focus count (how many times the window was brought to the foreground) at 0x08, and a cumulative focus duration in milliseconds at 0x0C. From there, forty-four bytes of intervening data lead up to offset 0x3C, where an eight-byte FILETIME records the last execution time in UTC, followed by four more bytes of flags. One quirk worth knowing before you trust a run count: the older 16-byte format used a five-count bias. That adjustment does not apply to modern 72-byte entries, so a modern value of five cannot be read as one run on that basis. The safest way to disambiguate is to cross-reference the focus count and the last-execution timestamp against other artifacts.

Two extra details are worth keeping in your back pocket. First, the `LastWriteTime` of the `Count\` key itself provides a secondary, corroborating timestamp, marking the last time any entry under that GUID changed, which is useful if someone has doctored an individual value's embedded FILETIME without realizing the key's own metadata gives them away. Second, UserAssist entries survive long after the program they describe has been uninstalled, so a UserAssist entry is evidence a program once ran, not evidence it's still present. The standard tools for reading this artifact are Eric Zimmermann's `RECmd.exe`, using its UserAssist batch mode, and Didier Stevens' dedicated `UserAssist.exe`.

CHAPTER 9

The MRU universe: recent documents, dialogs, and typed paths

Beyond UserAssist, Windows Explorer maintains an entire family of "most recently used," or MRU, lists, each one covering a slightly different kind of interaction. Individually they're small and easy to overlook. Together they form a detailed map of what a user opened, saved, typed, and searched for.

RecentApps and the task switcher

Two related keys capture interaction that happens outside the ordinary shell launch path. `HKCU\Software\Microsoft\Windows\CurrentVersion\Search\RecentApps\`records recent app activity on Windows 10 1607/1703: each app gets its own GUID-named subkey holding an `AppId` (the AUMID for a packaged app, or a plain path for a Win32 one), a `LastAccessTime` FILETIME, and a `LaunchCount`. Nested underneath, a `RecentItems` subkey can further enumerate the specific documents opened within that app from the task-switcher UI, each carrying its own path, timestamp, and display name. `RecentApps` without `UserAssist` does not prove a Search launch. Sources disagree on 1709; later builds generally do not populate this key.

A related but separate key, `HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\FeatureUsage\`, tracks taskbar interaction. Its `AppSwitched` subkey counts, per executable name, how many times that window

was selected through the taskbar, and `ShowJumpView` counts taskbar jump-list displays, such as a right-click on an app icon. Crucially, this doesn't require the window to have been launched through the shell at all: it can record a taskbar selection, no matter how the underlying process started, including processes launched by a service or scheduled task. That makes `FeatureUsage\AppSwitched` unusually useful for proving a window was visible and interacted with, even when its origin was something entirely non-interactive. The key's own `LastWriteTime` acts as a floor for the most recent interaction, though it doesn't pin down the exact moment.

The RecentDocs family

The core of the MRU universe is `HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs\`, which tracks files opened or saved through the standard Open/ Save dialog and Explorer itself. It's organized into a default list mixing all file types together, plus a separate subkey for each file extension the user has actually opened (`.docx`, `.pdf`, `.xlsx`, `.zip`, and so on), a `Folder` subkey for recently visited folders, and GUID-named entries for shell namespace items with no ordinary extension. Each per-extension subkey holds a series of numbered `REG_BINARY` values, one per recent file, storing the filename (not the full path) along with a serialized shell item ID or shortcut blob, and an ordering value called `MRUListEx`: a sequence of four-byte indices that tells you the actual most-recent-to-least-recent order, terminated by `FF FF FF FF`. The `LastWriteTime` of a given extension's subkey approximates when the most recent file of that type was accessed, and where shell items or related artifacts provide a path, it can reveal network

locations, removable media paths, and even FTP locations alongside ordinary local files.

Three related dialog-specific MRUs round out the picture. `ComDlg32\OpenSavePidlMRU\` records every path selected through a standard Open or Save dialog, again split by file extension. `ComDlg32\LastVisitedPidlMRU\` tracks, per application executable, the last folder that application's Open/Save dialog was pointed at, storing the process name alongside a serialized path to that folder. `RunMRU\` holds the literal text typed into the Win+R Run dialog, one value per command, with a simple string (rather than a binary array) called `MRUList` giving the order; each value carries a trailing `\1` that's purely decorative and should be stripped when reading it. Because attackers frequently reach for Win+R to launch `cmd.exe`, `powershell.exe`, `mshta.exe`, or `regsvr32` with suspicious flags, this key often contains a verbatim record of exactly what was typed, typos included. Rounding things out, `TypedPaths\` records paths and URLs typed directly into the Explorer address bar, with the key's `LastWriteTime` again standing in for the most recent entry's timestamp.

The rest of the MRU picture

A handful of further MRU locations exist for more specialized purposes. `WordWheelQuery` holds the last roughly twenty-five search terms typed into the Explorer search box. `Map Network Drive MRU` retains UNC paths used when mapping a network drive. `StreamMRU` tracks Explorer's own folder-stream history. Office applications keep a completely separate, per-application file MRU under `HKCU\Software\Microsoft\Office\16.0\*\File MRU`. And Internet Explorer's `TypedURLs` key under `HKCU\Software\Microsoft\Internet`

Explorer`\TypedURLs` preserves URLs typed into Internet Explorer; legacy Edge uses separate browser artifacts. None of these individually carries the weight of RecentDocs or Shellbags, but a thorough review of a machine's activity almost always means checking all of them, because a user determined to avoid one particular MRU list rarely thinks to avoid all of them at once.

CHAPTER 10

Shellbags: folders that remember you visited

Of every artifact in this book, Shellbags might be the most complete record of folder-browsing behavior Windows keeps, and it's also one of the least understood outside forensic circles. Folders a user opens in Explorer can leave traces here: local folders, folders on removable drives, network shares, even the contents of ZIP archives, and the trace persists even after the folder itself is deleted or the drive is disconnected.

Shellbags live in two places, and both matter. For the interactive user's own hive, they sit at `HKCU\Software\Microsoft\Windows\Shell\Bags\` and `HKCU\Software\Microsoft\Windows\Shell\BagMRU\` inside `NTUSER.DAT`. But the more complete record, covering Open/Save dialogs and the Desktop namespace, lives in a second, easy-to-miss hive called `UsrClass.dat`, found at `%LocalAppData%\Microsoft\Windows\UsrClass.dat`, under the equivalent path `HKCU\Software\Classes\Local Settings\Software\Microsoft\Windows\Shell\Bags\` and `...\BagMRU\`.

`BagMRU` is structured as a tree that mirrors the Windows shell namespace itself. Its root default value holds a serialized shell item, called a PIDL, representing the Desktop; each numbered subkey underneath represents one child folder, so subkey `0` might be "This PC," and a further subkey `0` underneath that might be the C: drive, and so on, descending exactly as deep as the user has actually navigated. Each of these PIDL values encodes rich

metadata about the folder it represents. For an ordinary filesystem folder, the encoded structure includes a type byte distinguishing a root-level folder from one on a non-root volume, the folder's attributes, its last-modified date and time in old MS-DOS date/time format, and an 8.3-style short name. Where the structure is large enough to include an extension block (identifiable by a signature such as `0xBEEF0004`), it also carries DOS/FAT-format creation and access timestamps, plus the folder's full long filename in Unicode.

A parallel `Bags\` hierarchy stores view settings rather than navigation history: for each numbered bag, a `Shell\` subkey records the chosen view mode (icons, small icons, list, details, or thumbnails), sort column and direction, and grouping settings, linked back to the corresponding `BagMRU` node through a `NodeSlot` value.

What makes Shellbags so valuable forensically is durability. There's no expiration built in: a folder visited once, five years ago, on a USB drive that's since been thrown away, can still have an entry sitting in the hive today, with the drive letter, volume label, and last-access time intact. Folders inside ZIP archives get their own distinct entries, tagged with a type byte specific to the shell's ZIP handling. Network shares generate entries recording the server name, share name, and subfolder path navigated, all of which persist even after the share is removed or the credentials expire. The `MRUListEx` ordering combined with a parent key's `LastWriteTime` can establish the sequence in which folders were visited within a session, but timestamps inside the PIDL describe the folder's own file times, not the visit. The standard tool for parsing both hives with full PIDL decoding is Zimmermann's `ShellBagsExplorer.exe`.

CHAPTER 11

LNK files and jump lists: shortcuts and taskbar menus that testify

Shortcuts and jump lists might be the most information-dense artifacts covered anywhere in this book, because both are built on the same underlying file format, one specifically designed to preserve a target file's identity even after that file is gone, moved, or renamed.

The LNK file format

Whenever a file is opened through Explorer, an Open dialog, or almost any standard Windows application, the shell creates or updates a `.lnk` shortcut file recording that access. These accumulate primarily in `%AppData%\Microsoft\Windows\Recent\`, with parallel collections for Office documents specifically, the Start Menu, and the public and per-user Desktop. Critically, this only happens through the shell: direct file I/O alone does not request a recent-item shortcut, though an application can explicitly call SHAddToRecentDocs.

Every LNK file opens with a fixed 76-byte header. After a header-size field and a CLSID that always identifies the file as a shell link, it stores a bitmask of flags describing what optional data follows, the target's file attributes, and then three FILETIME fields: the target's creation time, last access time, and last write time, all as they stood at the moment the shortcut was made or updated. This is worth sitting with for a second, because it's easy to misread: those three timestamps describe the *target file*, not the LNK file

itself, which means a LNK file can preserve a precise record of a file's original timestamps long after that file has been deleted and even after a new, different file has been created at the same path. The header rounds out with the target's file size, an icon index, a "show command" value (normal, maximized, or minimized), and an optional hotkey.

The flags bitmask in the header determines which optional blocks follow. A LinkInfo block, when present, carries either a local base path or, for files on network shares, a `CommonNetworkRelativeLink` structure. Embedded inside LinkInfo, a Volume ID structure records the drive type (fixed, removable, network, CD-ROM, and so on), the volume's serial number, and its label at the time the link was made, which becomes a direct pivot point for tying a shortcut to a specific physical drive, cross-referenced against the `MountedDevices` registry key and Shellbag volume entries covered elsewhere in this book. Beyond the core header and LinkInfo block, a LNK file can carry a whole series of optional "ExtraData" blocks, each identified by its own four-byte signature: an environment-variable block for paths that need expansion, a console-settings block recording font and color choices, a special-folder block, a Darwin block carrying an MSI install identifier, an icon-environment block, a shim-layer block recording which AppCompat shim applied, a property-store block, a known-folder block, and a full shell ID list block that's essentially always present on Windows Vista and later.

Of all these blocks, the most forensically valuable by far is the tracker data block. It's a fixed 88-byte structure carrying the NetBIOS name of the machine the shortcut was created on, plus two pairs of GUIDs: a volume GUID and a file object GUID reflecting the target file's identity at link-creation time, and a second "birth" pair preserving those same values from before any

subsequent moves or renames. That file object GUID corresponds directly to the NTFS `$OBJECT_ID` attribute discussed in part four, which means a LNK file can be used to trace a specific file's identity across renames and even across a move to a different machine, since the recorded machine name travels with the shortcut wherever it's later copied.

A few practical notes round this out. The LNK file's own timestamps (as opposed to the target's, embedded inside it) reflect filesystem activity, not necessarily a user opening the target. LNK files are not cleaned up automatically when their target is deleted, so `Recent\` accumulates a long tail of shortcuts pointing at things that no longer exist, which is often exactly what makes them useful. Whether this tracking happens at all is governed by `HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\Advanced\Start_TrackDocs`. The standard parsing tool, again from Zimmermann's suite, is `LECmd.exe`.

Jump lists

Jump lists are the per-application menus that appear when you right-click a taskbar icon, and under the hood they're really just structured collections of the same LNK data described above. Two kinds exist side by side. Automatic destinations, stored as `.automaticDestinations-ms` files in `%AppData%\Microsoft\Windows\Recent\AutomaticDestinations\`, are generated by Windows itself as you use an app. Custom destinations, stored as `.customDestinations-ms` files in the neighboring `CustomDestinations\` folder, hold items an application has explicitly pinned through its own jump-list API.

Each file is named after a sixteen-character hexadecimal hash of the owning application's identifier, computed by the shell; a

handful of these hashes are stable and well known, including 1b4dd67f29cb1962 for Windows Explorer, 9b9cdc69c1c24e2b for 64-bit Notepad, and 1bc392b8e104a00e for Windows 7 Remote Desktop Connection. Identifiers can vary; resolve unfamiliar hashes against a documented mapping.

Automatic destination files are OLE Compound File Binary containers, the same container format historically used by older Office document formats, identifiable by the byte signature `D0 CF 11 E0 A1 B1 1A E1`. Inside, a stream called `DestList` holds the ordering and metadata for every entry, while numbered streams (`1`, `2`, and so on) each hold one complete embedded LNK file, parseable exactly like a standalone shortcut. The `DestList` stream opens with a 32-byte header recording a version number, the highest entry number ever assigned, total and pinned entry counts, and a revision counter that increments on every change. Each entry that follows records a checksum, the same volume and object GUID pair seen in LNK tracker blocks, a NetBIOS address for remote files, the target's UTF-16LE path, a sequential entry number, a 32-bit pin status (`0xFFFFFFFF` meaning unpinned, nonnegative values indicating a pin position), an access count specific to that file, a FILETIME for the last access, and the originating machine's hostname. That access count is worth calling out on its own: it's more granular than anything UserAssist offers, since it's tracked per individual file rather than per application.

Custom destination files skip the compound-document format entirely in favor of a simple sequential layout: a count of entries, followed by each entry's complete raw LNK data, separated by a fixed four-byte marker (`0xBABFFBAB`) that also closes out the file.

Jump lists share LNK's durability: entries remain in `AutomaticDestinations\` long after the generating application has

been uninstalled, and the LastAccessTime recorded inside a `DestList` entry is tracked independently of the embedded LNK's own timestamps, occasionally capturing a more recent access than the shortcut itself shows. The standard parsing tool is `JLECmd.exe`, again from Zimmermann.

The Recent folder itself

Finally, the `%AppData%\Microsoft\Windows\Recent\` folder that holds all of this is itself worth understanding as a unit, since it's maintained by a specific shell API, `SHAddToRecentDocs()`. That function gets called by ordinary Explorer file opens, by Office applications, and by most Win32 programs that use the standard file-open dialog, but direct file I/O alone does not call it; applications may explicitly request recent-item updates, and packaged apps also have WinRT access-list mechanisms. `HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\MaxRecentDocs` controls the legacy Recent Items menu, whose default is ten items; it does not set a universal retention limit for files on disk. Because a LNK file here is updated, not replaced, on repeat access, its own last-modified timestamp effectively records the most recent time that particular file was opened, and the whole folder ties directly back to the `RecentDocs` registry key described earlier in this part, giving you two independent, cross-checkable records of the same underlying behavior.

PART FOUR

the file system itself

CHAPTER 12

The master file table and the two kinds of timestamps

Every file and folder on an NTFS volume has an entry in the Master File Table, or `$MFT`, a structure so fundamental to the file system that it isn't an ordinary user file; raw-volume access or `NTFS` control calls can retrieve its records. Each entry is a fixed-size record, ordinarily 1,024 bytes (or 4,096 bytes on volumes configured for large records), opening with the four-byte signature `FILE` and a short header that records, among other things, how many times that MFT slot has been reused, how many hard links point at the file, whether the record represents a file currently in use or a directory, and how much of the record's allocated space is actually occupied.

The interesting part isn't the header, though. It's the attributes that follow it, because NTFS stores two entirely separate sets of timestamps for every file, and understanding the difference between them turns out to matter a great deal in practice. The first, called `$STANDARD_INFORMATION`, holds four FILETIME values: when the file was created, when its data was last modified, when the MFT record itself was last changed, and when the file was last accessed. This is the set of timestamps that ordinary tools, and Explorer's own file properties dialog, show you. `SetFileTime`() can change creation, access, and write times, but not MFT change time, which makes it the target of choice for anti-forensic "timestomping," where an attacker deliberately backdates a file to blend it in with older, legitimate ones.

The second set lives in a separate attribute called `$FILE_NAME`, and it carries the same four timestamps again, this time alongside the file's parent directory reference, its allocated and actual size, its attribute flags, and the length and character-set namespace of its filename. Critically, these timestamps are set directly by the NTFS kernel driver rather than through a documented user-mode API, which makes them considerably harder to falsify; most casual timestomping tools never touch them at all, leaving a mismatch between the two attribute sets as a strong tell that tampering occurred. Investigators refer to this four-timestamp, two-attribute comparison informally by the shorthand MACB, for Modified, Accessed, Changed (meaning the MFT record), and Born (meaning created), and cross-checking `$STANDARD_INFORMATION` against `$FILE_NAME` is one of the standard first moves in any timeline analysis.

A third attribute, `$OBJECT_ID`, assigns a persistent 64-byte identity to a file: a present-day object ID, the GUID of the volume where the file was originally created, the object ID it was born with, and an optional domain ID. This is the same identifier that reappears inside a LNK file's tracker data block, described in the previous chapter, which is what allows an investigator to follow a specific file's identity across renames and moves even when its path and name have changed entirely. NTFS also maintains a directory-wide index of these object IDs at `\$Extend\$ObjId`, and a separate index of directory contents themselves, called `$I30`, implemented as a B-tree; because B-trees don't immediately reclaim space when an entry is removed, deleted directory entries can sometimes still be recovered from `$I30` slack space well after the fact.

The file's actual content lives in a fourth attribute, `$DATA`. Small files, roughly under 700 bytes, are stored resident, meaning their

content sits directly inside the MFT record itself with no separate disk allocation at all. Larger files are non-resident, and the MFT record instead holds a series of data runs, compact pairs recording a starting cluster and a run length, that together describe where on the volume the actual bytes live. One property of `$DATA` deserves special attention: a file can have more than one, through a feature called alternate data streams, where an additional named stream rides alongside the primary content. Windows itself uses this constantly. Supporting downloaders can attach a `Zone.Identifier` stream, recording a numeric zone (3 for Internet, 1 for a local intranet, 2 for a trusted site, 4 for restricted), plus the referring and hosting URLs the download came from, which is exactly how Windows knows to warn you before running a file you just downloaded. SmartScreen uses its own alternate stream to store security metadata, and, less benignly, malware has long used alternate data streams as a way to hide a payload inside what looks, at a glance, like an ordinary file.

CHAPTER 13

$LogFile, $UsnJrnl, and $Recycle.Bin

Beyond the MFT itself, NTFS maintains a pair of journals that record change over time, plus one more structure dedicated specifically to deletion. Together they're often what's left to examine once more visible traces have been cleared.

The USN change journal

The `$UsnJrnl`, sitting inside `\$Extend\$UsnJrnl` and accessible through the Win32 USN-journal control calls, is a running log of file-system changes: creates, deletes, renames, data extensions, and security changes. Repeated changes can share accumulated reason flags. Every `USN_RECORD_V2` entry carries a reference to the changed file and its parent directory, an ever-increasing sequence number called the USN, a FILETIME marking when the change occurred, a bitmask explaining what kind of change happened, and the changed file's name (though notably not its full path). That bitmask is detailed: separate flags exist for data being overwritten, extended, or truncated, for a named alternate stream being modified, for extended attributes or security descriptors changing, for a file being created or deleted outright, for the old and new names in a rename operation (recorded as two separate entries), for an object ID changing, and a closing flag marking the final entry associated with a closed file handle.

What makes the USN journal so useful is precisely that it survives the events it records: a file can be created and deleted within seconds, and while the file itself is gone without a trace, the journal still holds an entry showing it happened, along with the

filename and a precise timestamp. The journal isn't infinite, though; it's a rolling log sized by a companion `:$MAX` configuration stream, and older entries get overwritten as it fills, typically covering anywhere from a few days to a few weeks of activity depending on how busy the volume is and how it's been sized. Live systems can be queried with `fsutil` usn readjournal or the documented journal API to read retained records.

The transaction log

A separate structure, `$LogFile`, serves an entirely different purpose: it's NTFS's own crash-recovery mechanism, recording metadata changes (not file content changes) as redo and undo entries so the file system can repair itself after an unclean shutdown. Because it operates independently of `$UsnJrnl`, it can occasionally preserve evidence of low-level structural changes that happened between two `$UsnJrnl` entries, or after the change journal itself has rolled past the relevant window. Parsing it is more specialized than parsing the USN journal, and the community tool most often used for the job is Joakim Schicht's `LogFileParser`.

The Recycle Bin

Finally, deletion through Explorer's ordinary delete function (as opposed to permanent deletion) routes through `$Recycle.Bin`, found at the root of each participating volume and organized into one subfolder per user, named after that user's SID. Every deleted item there produces a pair of files sharing a matching random suffix: an `$I` file holding metadata, and an `$R` file holding the actual deleted content. The metadata file is a small, fixed structure recording a version number (version 2 on Windows 10 and later), the original

file's size, a FILETIME marking the exact moment of deletion, and, in version 2, the length and full original path of the deleted file. Of everything covered in this chapter, the Recycle Bin's `DeletionTime` field is often the single most precise timestamp available for pinning down exactly when a file was removed, which is why it remains one of the first places investigators check even in cases where a suspect believed they'd covered their tracks by emptying it.

PART FIVE

what the system saw

CHAPTER 14

Windows event logs, channel by channel

By default, Windows Event Logs use .evtx files under `%SystemRoot% \System32\winevt\Logs\`, one file per channel, and the operating system maintains dozens of these channels simultaneously, each focused on a different subsystem. Chapter 40 covers the binary format of an EVTX file itself in detail; this chapter is a map of the territory, a tour of which channel records what, so that later chapters diving deep into a specific channel have somewhere to point back to.

The three original, general-purpose logs are the ones most people have at least heard of. `Security.evtx` is the audit trail: successful and failed logons, group membership changes, process creation, scheduled task activity, and file share access, assuming the relevant audit policy is turned on. It's dense enough, and important enough, that it gets an entire chapter of its own later in this book (chapter 36), covering logon types, process-creation fields, and account-management events in full. `System.evtx` captures service installation, service state changes, and the boot and shutdown events that chapter 16 walks through in detail. `Application.evtx` catches application-level errors, Windows Error Reporting submission events, and installer activity, most of which is covered fully in the next chapter, on crash forensics.

Beyond those three, a long list of feature-specific channels each cover one particular subsystem. The PowerShell operational channel logs script block content and module activity, and gets its own full chapter (35) given how central it's become to both

legitimate administration and attacker tradecraft. The Task Scheduler operational channel records task registration, task actions starting and completing, and task deletion (event IDs 106, 200, 201, and 141 respectively), with a full treatment in chapter 26. The WMI-Activity operational channel is where persistent WMI subscriptions surface, specifically through event ID 5861, and chapter 34 covers why that matters. The BITS-Client operational channel logs job creation, transfer start, and transfer stop (event IDs 3, 59, and 60), relevant to chapter 33's discussion of BITS as a persistence mechanism. AppLocker's dedicated channels log allowed and blocked execution attempts, covered in chapter 36 alongside security auditing.

A few channels don't get a dedicated chapter elsewhere in this book and are worth covering here in full. If Sysmon, Microsoft's free System Monitor tool, has been deployed on a machine, its operational channel becomes one of the richest sources available, logging process, network, file, registry, and named-pipe activity including event IDs above 25, effectively turning a stock machine into something closer to a lightweight EDR sensor. The SMB client's security channel logs event ID 31017 when an insecure guest logon is rejected, relevant to any investigation touching lateral movement over file shares. An NTFS-specific operational channel captures lower-level file system errors and USN journal events, complementing the `$UsnJrnl` artifact covered in chapter 13. The Kernel-PnP configuration channel logs device configuration and start (event IDs 400 and 410), making it a primary source for USB and peripheral device forensics. A Remote Desktop-specific channel, `RemoteDesktopServices-RdpCoreTS`, logs accepted and established TCP connections (event IDs 131 and 98), while a related session-manager channel tracks logon, logoff, disconnect, and reconnect events for both console and RDP sessions. Group

Policy's own operational channel records GPO application activity generally, with the specific event IDs covered in chapter 39 alongside other Group Policy artifacts. And on domain-joined or MDM-managed machines, a dedicated enterprise diagnostics channel captures device enrollment and compliance events.

Taken as a set, these channels are Windows' most deliberate record-keeping: unlike a registry MRU list or a cache file that exists as a side effect of something else, an event log exists because someone at Microsoft decided that particular category of event was worth writing down on purpose. That deliberateness is also what makes tampering with `Security.evtx` specifically (via event ID 1102, an explicit "audit log was cleared" event) such a strong signal on its own; chapter 43 covers what other traces survive even after a log has been cleared.

CHAPTER 15

When Windows crashes: WER, minidumps, and kernel dumps

Chapter 1 introduced the services behind Windows Error Reporting. This chapter picks up where that left off, looking at what WER actually writes to disk, in what format, and what an investigator or a curious owner can learn from it.

Where crash reports live

Every crash report starts life in a queue. Per-user queues can be found at `%LocalAppData% \Microsoft\Windows\WER\ReportQueue\`, with each crash getting its own folder named something like `AppCrash_executablename_PID_timestamp_cabinet`. Inside that folder sits `Report.wer`, a text metadata file containing key=value fields; a `.mdmp` minidump, when dump collection is enabled and succeeds; occasionally a larger `.hdmp` heap dump if the system is configured to capture one; and an `appcompat.txt` file holding application compatibility context. Once a report has been uploaded or otherwise resolved, it moves to a parallel `ReportArchive\` folder. System-wide reports use the parallel store under `%ProgramData%\Microsoft\Windows\WER\`, which has its own queue, archive, and a working `Temp\` folder used while a report is being assembled.

Reading a Report.wer file

The `Report.wer` text file holds much of the useful, human-readable detail. `EventType` identifies the report type, while Sig`[`n`]`.Name and Sig`[`n`]`.Value pairs label the problem-signature fields. Other fields vary by report type and build; XML companion files may hold more OS and process metadata. For the most common case, `APPCRASH`, those parameters spell out the application name and version, the application's build timestamp, the name and version of the module that actually faulted, that module's own timestamp, an exception code, and the offset within the faulting module where the exception occurred.

The `EventType` value itself is worth knowing on sight, since it tells you immediately what category of failure you're looking at: `APPCRASH` for an ordinary unhandled exception, `BEX` or `BEX64` for a buffer-overflow exception (worth flagging as a potential exploitation attempt rather than an ordinary bug), `CLR20r3` or `CLRDIE` for .NET runtime failures, `MoAppCrash` for a UWP app crash, `LiveKernelEvent` for a live kernel dump taken while Windows can continue running, `WindowsUpdateFailure` for a failure specifically in an update component, and `InPageError` for a page fault caused by failing to read paged-out memory, often an early sign of failing disk hardware.

The named exception-code field is its own small vocabulary. `0xC0000005` is an access violation, the general-purpose "the program touched memory it shouldn't have" code, which can mean anything from a simple null-pointer bug to a shellcode crash. `0xC0000094` is a divide-by-zero. `0xC0000409` can also mark a deliberate fast-fail stop, not only a stack overrun caught by the

compiler's `/GS` protection and does not by itself establish exploitation. `0xC000001D` is an illegal instruction, sometimes the result of anti-debugging or virtual-machine-detection code misfiring. `0xC0000374` indicates heap corruption, frequently associated with heap-spray exploitation techniques. `0x80000003` is simply a software breakpoint left behind in code, often a sign a debugger was attached or that debug code shipped by accident. `0xE06D7363` identifies Microsoft C++ exceptions, while `0xE0434352` identifies CLR/.NET exceptions; neither alone establishes an exploitation attempt.

Kernel and system-wide dumps

Application crashes are only half the picture. When the kernel itself faults, Windows produces a very different kind of artifact under `%SystemRoot%\Minidump\`, named by date and a sequence number, alongside an optional full memory dump written to `%SystemRoot% \ MEMORY.DMP` if the system is configured for it. Per-user application dumps, when explicitly configured, land in `%LocalAppData% \ CrashDumps\`. What actually gets captured is controlled by a set of registry values under `HKLM\SYSTEM\CurrentControlSet\Control\ CrashControl\`: `CrashDumpEnabled` chooses between no dump, a complete dump, a kernel-only dump, a small minidump, or an "automatic" kernel-dump mode that lets Windows adjust paging-file size, while `DumpFile` and `MinidumpDir` set the output locations, `AutoReboot` controls whether the machine restarts automatically after a bugcheck, and `LogEvent` decides whether a System event gets written at crash time.

User-mode minidumps use `MINIDUMP`; kernel small dumps use a different format. The user-mode header

opens with the signature `MDMP`, followed by a version marker, a count of the data streams the file contains, an offset to the directory listing those streams, an optional checksum, a timestamp recorded as an ordinary Unix time, and a flags field describing exactly what categories of data the dump includes. What makes a minidump forensically valuable is what those streams typically contain: the full call stack of every thread at the moment of the crash, the exception context including CPU register values, a complete list of loaded modules with their base addresses, and often the Process Environment Block, which holds the process's command line and environment variables. That module list, cross-referenced against expected DLLs, is frequently how retrospective DLL injection gets discovered: an unexpected module, loaded at an unexpected base address, sitting in a crash dump from months ago.

Turning it down, and reading what it wrote

WER's behavior is itself governed by registry values under `HKLM\SOFTWARE\Microsoft\Windows\Windows Error Reporting\`: `Disabled` turns the whole system off, `DontSendAdditionalData` suppresses additional, second-level report data, and `LoggingDisabled` disables `WER` logging. A `LocalDumps` subkey allows per-application overrides of the dump folder, retention count, and dump type, and a separate `DebugApplications` subkey controls whether users must choose Debug or Continue for a named executable. Whatever configuration is in effect, WER logs its own activity through Microsoft-Windows-WER-Diag and in `Application.evtx`, where event ID 1000 marks the underlying application error, 1001 the WER fault-bucket metadata, and 1002 an application hang.

CHAPTER 16

Problem Steps Recorder, boot and shutdown events, hibernation, and the page file

This chapter closes out part five with a set of artifacts unified less by subsystem and more by what they capture: moments of transition, whether that's a user manually documenting a bug, a machine starting up or shutting down, or the system freezing its own memory state to disk.

Problem Steps Recorder

Buried in Windows since Windows 7, `psr.exe` is a small utility that records a step-by-step account of what a user does on screen, producing a ZIP archive at a path and with a filename the user chooses. Inside, an MHTML file embeds base64-encoded PNG screenshots of each recorded step, and XML embedded in the MHTML report lists the actual event timeline: each `UserActionEvent` entry carries an action index, a precise timestamp, an event type like a left click, a plain-language description ("User left clicked on ... in ... on screen 1"), a bounding rectangle marking where on screen the action happened, and a reference to the associated screenshot.

PSR deliberately does not capture actual keystrokes, only descriptions like "User typed into field," but those descriptions frequently include the name of the UI element involved, which can be revealing on its own: a description like "User typed into 'Password' field in 'MyBank Login'" tells you a great deal even

without the literal characters typed. PSR output can show up in an investigation two very different ways: as a legitimate helpdesk artifact from a pre-incident troubleshooting session, or, less innocently, as documentation an attacker created of their own steps. Since PSR has no dedicated registry storage of its own, the way to find evidence it ran at all is to check the other artifacts in this book, particularly LNK files, jump lists, and RecentDocs, for references to `psr.exe` and to the ZIP files it produced. Its behavior can be disabled outright via `HKLM\SOFTWARE\Policies\Microsoft\Windows\appcompat\DisablePSR`.

Boot and shutdown

A predictable set of System.evtx events marks every transition a machine goes through. Event ID 6005 marks the event log service starting, which is the practical definition of a boot; 6006 marks it stopping, meaning a clean shutdown; 6008 flags that the previous shutdown was unexpected, in other words a crash or power loss; 6009 logs the Windows version and build at startup; and 6013 records system uptime in seconds since the last boot. A separate event source, Kernel-Power, logs event ID 41 whenever the system comes back up without having gone through a clean shutdown first, which is often the more reliable of the two "did this crash" indicators. Two further events, sourced from User32, capture shutdown intent directly: event ID 1074 fires whenever a process or user initiates a shutdown or restart, and its six parameters spell out exactly which process requested it, which machine and user account made the request, a stated reason string, a numeric reason code (a scheduled restart and an operating-system patch restart carry different codes), what type of shutdown was requested, and any free-text comment supplied. Event ID 1076 records the reason given for the most recent shutdown after the fact.

Hibernation, the page file, and suspended app memory

Three files sitting at the root of the system drive capture the machine's memory state in different ways. `hiberfil.sys` holds a compressed image of kernel memory, using the same XPRESS Huffman compression scheme as prefetch files, written whenever the system hibernates. Since Windows 8, ordinary shutdown itself often triggers a partial version of this through the "fast startup" feature, sometimes called hiberboot, which preserves only kernel and driver state rather than a full memory image; a genuine hibernate action captures everything. Because it's fundamentally a memory snapshot, `hiberfil.sys` can hold the running process list, active network connections, encryption keys still resident in memory, open file handles, and loaded DLLs, all as they stood at the exact moment of hibernation, and memory-forensics tools like Arsenal Recon's Hibernation Recon can extract supported formats; Volatility 3 requires suitable hibernation-layer or conversion support. Whether hibernation is available at all is controlled by `HKLM\SYSTEM\CurrentControlSet\Control\Power\HibernateEnabled`.

`pagefile.sys`, the ordinary virtual memory backing store, holds a messier but often just as revealing record: fragments of whatever memory got paged out over time, including process stack data, heap contents, and network buffers. Because it accumulates gradually rather than capturing one clean snapshot, the most productive way to examine it is often a simple strings search, which regularly turns up command lines, URLs, and fragments of decrypted data that were briefly resident in memory at some point. A third file, `swapfile.sys`, plays a narrower role in reclaiming memory from suspended packaged apps: it holds the memory of suspended UWP applications, giving investigators a similar window into app state for the packaged-app ecosystem that `pagefile.sys` provides for traditional processes.

Two final registry values round out this picture of system-level timing. `HKLM\SYSTEM\CurrentControlSet\Control\Windows\ShutdownTime` stores the FILETIME of the machine's last clean shutdown directly, and `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\InstallDate` records, as an ordinary Unix timestamp, exactly when Windows itself was installed on the machine, which is often the very first data point an investigator establishes before building out a timeline of everything that happened afterward.

PART SIX

the shell remembers

CHAPTER 17

Thumbnails, icons, and the Windows search index

Thumbnail and icon caches

Every time Explorer needs to show you a thumbnail preview or an application icon, it checks a set of cache files before regenerating the image from scratch, and those cache files, sitting under `%LocalAppData%\Microsoft\Windows\Explorer\`, accumulate a visual record of files you've browsed. Windows keeps a separate cache file for each thumbnail size it commonly needs: 16, 32, 48, 96, and 256 pixels for the small-to-large icon views, and 768, 1024, 1280, 1920, and 2560 pixels for jumbo icon views, high-DPI displays, 4K displays, and ultra-wide monitors respectively. An index file, `thumbcache_idx.db`, maps thumbnail identifiers to cache entries; it is not a complete path-to-image history. A few more specialized caches round things out: `thumbcache_wide.db` and `thumbcache_wide_alternate.db` for panoramic images, `thumbcache_exif.db` for thumbnails extracted from embedded EXIF data, `thumbcache_sr.db` for video frame thumbnails, and a family of `iconcache_*.db` files holding application icon bitmaps at various sizes.

The forensic value here is straightforward but important: a thumbnail can outlive its source, without proving the user viewed it, even after the source file itself has been deleted. Entries hold cache identifiers, sizes, checksums, and image data; finding source paths and times needs other evidence, which means that in the right circumstances, an

investigator can recover a visual preview of a file that no longer exists anywhere else on the system.

The Windows Search index

A separate and much larger artifact, the Windows Search index, lives at `%ProgramData%\Microsoft\Search\Data\Applications\Windows\Windows.edb` (ESE) or `Windows.db` (SQLite), and is maintained by the `WSearch` service through a trio of processes: `SearchIndexer.exe` running the core engine, and `SearchProtocolHost.exe` and `SearchFilterHost.exe` handling protocol-specific crawling and file-format-specific content extraction respectively. Its configuration lives at `HKLM\SOFTWARE\Microsoft\Windows Search\`, including a `DataDirectory` value pointing at the index location, and a separate `CrawlScopeManager` key that defines exactly which locations get indexed in the first place.

What actually gets indexed is extensive: the full path, filename, last-modified date, and size of every indexed file, author and title metadata pulled from document properties, and, for supported formats like DOCX, PDF, plain text, and connected email accounts, searchable text extracted from the file. For a mailbox connected through Outlook via MAPI, the index additionally captures sender, recipient, subject, and date for individual messages. ESE versions use property tables such as `SystemIndex_0A` or `SystemIndex_PropertyStore`, gathering tables such as `SystemIndex_Gthr`, and the `MSysObjects` catalog. SQLite versions use `SystemIndex_1_PropertyStore` and its metadata table.

The detail worth remembering is that the search index doesn't necessarily stay in sync with the file system in real time. If a file was indexed before it was deleted, its title, metadata, and even full text

content can persist in `Windows.edb` well after the file itself is gone and before the index has been re-crawled to notice, making the search index, like the thumbnail cache before it, a source of evidence for files that technically no longer exist anywhere on disk.

A separate, less documented file called `WaitList.dat` supports input personalization. Original research on Windows 8.1 found text and metadata from emails, contacts, and documents harvested through the search indexer, including content whose source was no longer available. Its location was `%LocalAppData%\Microsoft\InputPersonalization\TextHarvester\.` It is not merely a people-frequency ranking: the retained text itself can be forensic evidence. Availability and contents depend on the system's version and personalization settings.

CHAPTER 18

Windows Timeline and the activities cache

Windows Timeline, the feature that once let you scroll back through a visual history of recent activity across your devices, has been deprecated as a user-facing interface, but the database underneath it is still very much alive, still populated by the Connected Devices Platform in the background regardless of whether anyone ever opens the Timeline view itself.

The core database, `ActivitiesCache.db`, is an ordinary SQLite3 file living under `%LocalAppData%\ConnectedDevicesPlatform\`, inside a folder named after a per-user or per-platform hash assigned when the Connected Devices Platform service, `CDPSvc` and its per-user counterpart `CDPUserSvc`, first initializes. Older activity-sync configurations sent data to `activityfeed.microsoft.com`, and local sync state is tracked at `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\ActivityFeed\`.

Its central `Activity` table is one of the richer schemas in this book. Each row carries an activity UUID, an app-specific activity identifier, an activity type code, a status flag (active, updated, deleted, or ignored), an optional parent activity ID for grouped items, arbitrary tag and group fields, a cross-device correlation ID, last-modified and expiration timestamps, a JSON payload holding the actual content of the activity, a priority value, a flag for whether the activity stayed local or synced to the cloud, the originating device's GUID, a cloud-creation timestamp where present, separate start and end timestamps, a client-side last-modified timestamp, a clipboard-specific payload

size field, an enterprise tenant ID for organizational accounts, and a cloud sync ETag. The activity type codes themselves are worth a caveat: research into this schema across different builds has documented some inconsistency in exactly what a few numeric codes mean, though the WindowsTimeline research maps 10 to clipboard text and 16 to copy/paste, with 11 and 12 covering Windows system operations. Other codes, including 5, 6, 16, and 23, have been documented representing different things (app activation versus a general file or app open, a legacy Edge browsing entry versus nothing at all, an in-focus event versus a clipboard cloud-sync event, and an upload versus in-app navigation) depending on the Windows build and the source consulted, so treat any specific mapping for those codes as something to verify against the build you're actually looking at rather than as fixed law.

For a file-open activity specifically, the JSON payload typically includes a display name, the full path presented both as a description and a content URI, a type field identifying it as a file-open activation, an icon reference, and a background color used in the old Timeline UI. A companion `Activity_PackageId` table maps each activity back to the specific application and platform responsible for it, recording the originating device's hostname in an `ActivitySourceHost` field.

A few practical notes make this database considerably more useful in practice. It's an ordinary SQLite3 file, parseable with any SQLite library or with Zimmermann's dedicated `WxTCmd.exe`. The `StartTime` and `EndTime` columns describe an activity interval recorded through the app; they do not prove continuous use by a person. Deleted activities aren't necessarily gone: they're marked with an `ActivityStatus` of 3, and unallocated SQLite pages can sometimes still hold their row data. Because cloud sync writes activity from other paired devices back into this same local

database, it can reveal cross-device access patterns that go well beyond what happened on the machine you're examining. And because SQLite databases in active use are often accompanied by write-ahead-log and shared-memory files, `ActivitiesCache.db-wal` and `ActivitiesCache.db-shm`, any collection of this artifact should grab all three files together, since the WAL file can hold committed data that hasn't yet been merged into the main database file.

Sitting alongside `ActivitiesCache.db` in the same folder is a second, related database, `cdp.db`, which tracks the Connected Devices Platform's own state rather than individual activities. Its `Endpoints` table records every device ever paired with the machine through CDP, whether that's a phone through the Phone Link app, another Windows PC through Timeline sharing, or an Xbox console, giving each one a device ID, a friendly name, a protocol (Bluetooth Low Energy, IP, or cloud), and a `LastSeenTime` marking the most recent contact. A `Channels` table tracks which specific app is using which communication channel with a given endpoint, and a `UserData` table holds a JSON blob of Microsoft account identity state and device associations. Between the two databases, an investigator gets both a record of specific user activities and a record of exactly which other devices this machine has been talking to. Two related, narrower features worth knowing about: Phone Link, when paired with an Android phone, keeps its own local SQLite database of SMS messages and notifications under its Microsoft.YourPhone app package under `%LocalAppData%\Packages\`, while Nearby Share's device-discovery broadcasts over Bluetooth and Wi-Fi are session-only and leave no persistent local record of their own.

CHAPTER 19

Microsoft Edge's local footprint

Microsoft Edge, being a full Chromium-based browser, keeps a local footprint that's both far more detailed and far more standardized than most of the artifacts in this book, since much of its structure is shared with every other Chromium-derived browser.

Almost everything lives under `%LocalAppData% \Microsoft\Edge\User Data\`. A top-level `Local State` file, in JSON, holds global configuration and metrics that apply across every profile, including a `user_experience_metrics` block recording whether metrics collection is enabled, a client ID, a session ID, and a stability sub-block tracking crash counts for the browser process and for individual renderer processes, plus a nested Edge-specific telemetry level. Below that sits a `Default` folder (or a similarly named folder per additional profile) containing the actual browsing data: a `History` SQLite database, a `Web Data` database holding autofill and form data, `Login Data` and the newer `Login Data For Account` databases holding DPAPI-encrypted saved credentials (the second syncing through a Microsoft account), a Cookies database, found under Network\Cookies on newer builds, a `Bookmarks` file in JSON, a `Preferences` file (JSON, holding ordinary settings) alongside a tamper-evident, HMAC-signed `Secure Preferences` file, a `Favicons` database (useful precisely because icons can survive cleared history, without proving a visit), a `Sessions` folder preserving open tabs from the moment of a crash, folders holding per-extension local and session storage in LevelDB format, a folder of submitted Edge Feedback reports, DevTools session artifacts, a LevelDB-backed sync data

cache, and, at the top level, an `EVT` folder holding Edge's own internal event logs.

The `History` database's schema is worth walking through directly, since so much useful detail lives in its column definitions. A `urls` table holds one row per unique URL, tracking its title, a running visit count, a count of how many times it was actually typed rather than clicked, the timestamp of its last visit, and a hidden flag. A `visits` table logs each individual visit separately, linked back to its URL, recording the visit's own timestamp, a reference to whatever visit referred the user there, a transition type describing how the navigation happened, and how long the user actually spent on the page. A `keyword_search_terms` table captures search-engine queries, preserving both the original and lowercased form of each term. And a `downloads` table is its own small goldmine: a GUID, the file's current and originally intended paths, start and end timestamps, bytes received versus total expected bytes, a state field (in progress, complete, or cancelled), a danger classification, an interrupt reason, a SHA-256 hash of the downloaded file, the referring page, the URL of the tab that triggered the download, the HTTP method used, and both the reported and actual MIME type. The History database timestamps described here use the same epoch as FILETIME (midnight, January 1, 1601) but measured in microseconds rather than 100-nanosecond ticks, so converting one to a standard Unix timestamp means dividing by one million before subtracting the usual 11,644,473,600-second offset.

Edge also emits its own ETW events through a dedicated provider, registered under `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\WINEVT\Publishers\`, covering navigation events, crash reports, performance markers, and policy state changes; on some telemetry configurations, the

DiagTrack autologger session covered in part seven subscribes to this provider at its most verbose logging level. When Edge itself crashes, it writes minidumps and JSON metadata to a `Crashpad\reports\` folder inside its user data directory, and if the user submits feedback through the browser's built-in tool, that submission is staged locally as a screenshot and a `diagnostics.json` file before being uploaded, through either the Edge update process or a dedicated crash handler service, to Microsoft's error-recovery endpoint.

Separately from all this local file structure, Edge participates in the same account-level sync system covered in chapter 2, pushing favorites, full browsing history (if sync is enabled), saved passwords, open tabs across devices, autofill data, the list of installed extensions, and general browser settings up to a Microsoft account through `sync.microsoft.com`, with separate telemetry flowing to `data.microsoft.com`.

CHAPTER 20

Notifications, Cortana, Internet Explorer's legacy cache, maps, clipboard, and sticky notes

This chapter gathers a handful of smaller, self-contained artifacts that don't warrant a part of their own but are each worth knowing about.

Notifications and the Cortana databases

Windows can retain toast, badge, and tile notifications temporarily in a SQLite database called `wpndatabase.db`, sitting at `%LocalAppData%\Microsoft\Windows\Notifications\wpndatabase.db`. Its `Notification` table records each individual notification's handler, an activity ID, a type such as toast, badge, tile, or toastCondensed, the actual notification payload as an XML blob, when it arrived and when it was set to expire. `NotificationHandler` maps handler IDs to app identities and records registration times; WNSPushChannel holds channel details. The forensic value here can be considerable: a notification payload frequently preserves an email's subject line and sender, a preview of a chat message, a news headline and its URL, or calendar event details, sometimes surviving even after the originating application's own data has been cleared. Cortana, where it's still present, keeps its own ESE-backed database under a package-specific folder in `%LocalAppData%\Packages\`, with both a legacy path

(`Microsoft.Windows.Cortana_*`) and a newer one (`Microsoft.549981C3F5F10_*`) depending on the Windows build.

The Internet Explorer legacy cache

Even on machines that have never intentionally used Internet Explorer, a legacy cache called `WebCacheV01.dat` often still exists at `%LocalAppData%\Microsoft\Windows\WebCache\`, built in the same ESE format as several other artifacts in this book, retaining roughly ten days of history by default on Windows 8 and later. Inside, separate logical containers hold browsed URLs with titles, visit counts, and timestamps; cookies with their domain, value, and expiry; cached HTTP response bodies and headers; a record of downloaded files and their source URLs; HTML5 localStorage data; and RSS and Atom feed subscriptions.

Clipboard, maps, and sticky notes

Chapter 2 introduced the Clipboard User Service and its role in cloud sync; this is where its local footprint lives in detail. Clipboard history is stored under `%LocalAppData%\Microsoft\Windows\Clipboard\`, one GUID-named folder per clipboard item, each holding a small JSON metadata file recording the content types present, a timestamp, whether the item has been pinned, and its source, alongside a separate payload file named for its data-type GUID, holding the actual content: UTF-16LE text for plain text, RTF for rich text, or DIB and PNG data for images. Two registry values, `EnableClipboardHistory` and `SyncClipboardToCloud`, control whether local history and cloud sync are active respectively, and can both be disabled machine-wide through Group Policy values `AllowClipboardHistory` and

`AllowCrossDeviceClipboard`. When cloud sync is active, content flows from `cbdhsvc` through `CDPSvc` and `OneSyncSvc` to a Microsoft-account-authenticated endpoint, staging locally in the same `ConnectedDevicesPlatform` folder covered in chapter 18 along the way; local history supports up to 25 items of up to 4 MB each; cross-device sync supports text below 100 KB, while a "history only" setting keeps the same local record without ever syncing it anywhere. Clipboard activity also generates its own event log entries under `Microsoft-Windows-Clipboard-User/Operational`, distinguishing a plain history addition from a full clear or a cloud sync event.

Two smaller artifacts round out this chapter. The Windows Maps app keeps its offline map cache and state under a package-specific folder in `%LocalAppData% \Packages\Microsoft.WindowsMaps_*`. And the Sticky Notes app stores every note in a SQLite database called `plum.sqlite`, under its own package folder, where a `Note` table records each note's identifier, its text content, its on-screen window position, when it was created, whether it's been deleted, and a parent-note reference for grouped notes, while a companion Media table references images stored in the app's local media folder.

PART SEVEN

under the hood

CHAPTER 21

How the kernel watches every process, thread, and image load

Deep inside `ntoskrnl.exe`, the Windows kernel exposes a small family of notification routines that let a driver register a callback function and be told, synchronously, every time something fundamental happens: a process is created or destroyed, a thread starts or ends, or an executable image gets mapped into memory. These aren't Microsoft telemetry mechanisms in the way DiagTrack is; they're a general-purpose kernel facility that both Microsoft's own components and third-party security software rely on constantly.

Process creation and termination

The process-level hook comes in two forms. `PsSetCreateProcessNotifyRoutine` is the older, more limited version; `PsSetCreateProcessNotifyRoutineEx` is the modern variant, and it's the one worth understanding closely, because it's the version capable of actually denying a process creation outright rather than merely observing it. Both register a callback into a shared kernel array, `PspCreateProcessNotifyRoutine`, which holds up to sixty-four registered callbacks at once. When the Ex variant's callback fires, it receives the new process object, its process ID, and, for a creation event (as opposed to an exit), a structure called `PS_CREATE_NOTIFY_INFO`. That structure carries flags indicating whether a file-open name is available and whether the new process is a subsystem process, the parent process ID, the client ID of the thread that's actually doing the creating, a pointer to the image's

file object, the image's NT-format path, its full command line, and, critically, a writable `CreationStatus` field: setting it to an error status is how a callback denies process creation.

A handful of behaviors are worth keeping straight. The callback executes in the context of the thread that's doing the creating, not in the context of the new process itself. Only the Ex variant can deny creation. The callback fires before the new process's primary thread ever starts executing, which is what makes it useful as a genuine gatekeeping mechanism rather than just an after-the-fact log. And on process exit, rather than creation, the `CreateInfo` structure is simply null, with the process ID field identifying the process that just terminated. On a stock Windows 10 or 11 install, the best-known consumer of this hook is Windows Defender's own kernel driver, `WdFilter.sys`, also a filesystem minifilter at altitude 328010, alongside internal use by the kernel itself for job-object accounting and critical-process tracking, and, on domain-joined machines, sometimes `cng.sys` for cryptographic policy enforcement. Separately from any of these registered callbacks, ETW itself fires an event on every process creation and exit through the `Microsoft-Windows-Kernel-Process` provider, entirely independently of whatever drivers happen to be watching through this API. Investigators working from a memory image can enumerate the registered callback array directly in WinDbg, or use community tools like NotifyRoutineView or Volatility3's `windows.callbacks` plugin to the same end.

Thread creation and image loading

A parallel pair of routines, `PsSetCreateThreadNotifyRoutine` and its less-documented Ex counterpart, work the same way for thread creation and termination, registering into their own sixty-four-

entry array, `PspCreateThreadNotifyRoutine`. The documented callback fires on every thread created or destroyed system-wide, which on a busy machine is an extremely high-frequency event, and it receives only the owning process ID, the thread ID, and a boolean indicating creation versus termination, deliberately without a start address or function pointer. The documented Ex registration API, available since Windows 10, takes a notification type. Its callback still receives process ID, thread ID, and a create/delete flag, without a start address. This hook is a favorite of EDR and antivirus products specifically because it's one of the most reliable ways to catch remote thread injection, where an external process calls something like `CreateRemoteThread` or `NtCreateThreadEx` to plant a thread inside another process.

A third routine, `PsSetLoadImageNotifyRoutine`, fires when executable images are mapped into a process or kernel space (reported with process ID zero). `SEC_IMAGE_NO_EXECUTE` mappings are excluded. Its callback receives the image's NT device path, the target process ID, and an `IMAGE_INFO` structure describing the mapping in detail: the addressing mode, whether the image was mapped into kernel space, ImageMappedToAllPids (documented as always zero), the image's base address and size, and, importantly, a four-bit signature level and three-bit signature type field describing whether and how the image was code-signed. This hook is what allows kernel-level detection of DLL injection without relying on any user-mode hooking at all, and the signature-level field specifically is how a security product can flag an unsigned DLL loading into an otherwise fully signed, trusted process, a pattern that shows up constantly in real intrusions.

CHAPTER 22

Registry callbacks, object callbacks, and minifilters

Beyond process and thread creation, the kernel exposes three more hook families, covering registry access, object handle operations, and the file system itself.

Watching the registry

`CmRegisterCallbackEx` lets a driver intercept registry operations directly, in real time, whether to monitor, audit, or outright block reads and writes. Registration takes a callback function, an "altitude" string identifying the driver's position in the load order (the same altitude concept used by file system minifilters, covered later in this chapter), a pointer to the driver object, an optional context value, and returns a cookie used to unregister later. When the callback fires, it receives a `REG_NOTIFY_CLASS` value identifying the specific operation and a pointer to an operation-specific structure. Windows defines both "pre" notifications, which fire before the kernel actually performs the operation and can still be blocked, and "post" notifications, which fire afterward and can modify output parameters or the returned status: pre-notification classes include key deletion, setting a value, deleting a value, querying a key, opening a key, creating a key, and renaming a key, each with its own numeric class identifier, while post-notification classes mirror several of the same operations after the fact. For an operation like setting a value, the accompanying structure hands the callback the registry key object itself, the value's name, its type, a pointer to the actual data being written, and the size of that data.

The altitude string matters because it determines execution order: a callback registered at a higher altitude runs earlier, closer to the top of the callback chain, and antivirus products conventionally register somewhere in the 320000 to 329000 range. From a memory image, these registered callbacks can be enumerated by walking the kernel's internal callback list directly through WinDbg.

Object handles and the file system

A related hook, `ObRegisterCallbacks`, intercepts operations on process and thread handles themselves rather than registry keys, specifying which object type to watch (process or thread objects) and which operations to catch (a new handle being created, or an existing one being duplicated). Its pre-operation callback can actively reduce the access rights being granted before a handle is ever returned to the caller. This is the mechanism EDR and antivirus products rely on to protect themselves from tampering: by registering here, they can strip the rights needed to read another process's memory, which is specifically why malware attempting to dump credentials out of LSASS has to find a way around this callback rather than simply opening a handle to it directly.

The file system gets its own, more structured framework, built around the Filter Manager (`fltmgr.sys`) and a registration function called `FltRegisterFilter`. Installed minifilter drivers register under `HKLM\SYSTEM\CurrentControlSet\Services\{FilterDriverName}\Instances\`, each instance recording its own altitude and a flags value controlling whether it auto-attaches to every volume. Microsoft publishes conventional altitude ranges for different categories of filter: the highest FSFilter group sits near the

top, from 400000 to 409999 for FSFilter Top; 320000 to 329999 is the FSFilter AntiVirus range, with Windows Defender's own `WdFilter.sys` sitting at a specific, well-known altitude of 328010. Below that sit ranges reserved for replication, continuous backup, quota management, and compression, and `FileInfo.sys` has a legacy assigned altitude of 40500 and a newer assignment of 360500.5. It supplies file information; `NTFS` maintains the MFT. Lower-altitude filters can also exist.

CHAPTER 23

ETW, properly explained

Nearly every autologger, provider, and trace mentioned elsewhere in this book depends on the same underlying mechanism: Event Tracing for Windows, or ETW, a kernel-integrated logging framework that's been part of Windows for decades and that Microsoft's own telemetry pipeline is built entirely on top of.

ETW defines three roles. A controller starts and stops tracing sessions, subscribes specific providers to them, and configures buffer sizes, either from the kernel through an API called `NtTraceControl` or from user mode through the `StartTrace` and `ControlTrace` functions. A provider is any component that emits events, registered through a call to `EventRegister`, with manifest-based providers registered under `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\WINEVT\Publishers\`. A consumer receives those events, either in real time or by reading them back out of a saved `.etl` file, using the `OpenTrace` and `ProcessTrace` APIs.

Most ETW sessions start and stop as needed, but a special category called autologger sessions gets started by the kernel automatically at boot, before any user-mode service has even loaded, using configuration read entirely from the registry. That configuration lives under `HKLM\SYSTEM\CurrentControlSet\Control\WMI\Autologger\`, with one subkey per session. Each session's values spell out exactly how it behaves: whether it starts at boot at all, its session GUID, an output file path (left empty for sessions that never write to disk), a maximum file size, a flush interval, buffer sizing parameters, a clock

source (the high-precision QPC counter, ordinary system time, or CPU cycle counts), and a status value the kernel writes back after attempting to start it. The single most informative of these values is `LogFileMode`, a bitmask: a session can write sequentially to a file, write circularly and overwrite its oldest data, deliver events to a consumer in real time, buffer everything in memory without ever flushing to disk until read, run as a kernel-mode "system logger" session, remain independent of provider enable-state changes, or skip per-processor buffering in favor of a single shared buffer, and any given session typically combines several of these flags at once. Beneath each session's own key, a further layer of subkeys, one per subscribed provider, records whether that provider is currently enabled, a provider-specific keyword bitmask narrowing down which categories of event it should emit, a verbosity level running from 1 (critical only) to 5 (fully verbose), and an optional keyword filter requiring an exact match. On a running system, this whole apparatus is usually inspected and controlled through a small set of standard command-line tools: `logman` for querying and configuring sessions, `xperf` and `wpr` for capturing traces, and `tracerpt` for converting a captured `.etl` file into something human-readable.

CHAPTER 24

The autologgers that start before you log in, and the providers they subscribe to

Knowing the general shape of ETW autologgers is one thing; knowing which specific ones actually run on a stock Windows machine, and what they're each watching, is what makes the mechanism concrete.

The core boot-persistent sessions

The most consequential of these is `AutoLogger-Diagtrack-Listener`, one source for `DiagTrack`'s telemetry pipeline. In BSI's Windows 10 1607 analysis, it wrote to `%ProgramData%\Microsoft\Diagnosis\ETLLogs\AutoLogger\AutoLogger-Diagtrack-Listener.etl` before `DiagTrack` read and removed the file. The live Diagtrack-Listener session delivered events in real time. Logging mode depends on configuration and build. The number and choice of providers vary with build, diagnostic-data level, and system state; among the most consequential are `Microsoft-Windows-Kernel-Process` for process activity, `Microsoft-Windows-Kernel-Network` for network events (with verbosity set by configuration), `Microsoft-Windows-Compatibility-AppCompat` and `Microsoft-Windows-Compat-Appraiser` for compatibility telemetry, `Microsoft-Windows-Kernel-Power` for power state changes, `Microsoft-Windows-PowerShell` for script activity, `Microsoft-Windows-Shell-Core` for shell interaction,

`Microsoft-Windows-AppXDeployment` for packaged-app installation, and `Microsoft-Windows-Search` for indexing activity.

A second core session, the Circular Kernel Context Logger, often abbreviated CKCL, runs as a genuinely circular, buffering, in-memory session with no persistent output file under normal conditions, with its size set by configuration, continuously overwriting its oldest data. It's consumed on demand, either by Windows Error Reporting when a crash occurs or by `wpr.exe` when someone explicitly captures a performance trace, and it subscribes to kernel process, thread, disk, and graphics-driver providers. A third, `DiagLog`, writes its output to `%SystemRoot%\System32\LogFiles\WMI\DiagLog.etl` and its exact configuration varies somewhat across Windows builds.

Three more sessions bridge ETW directly into the ordinary Windows Event Log system you're used to seeing in Event Viewer: `EventLog-Security`, `EventLog-Application`, and `EventLog-System` each run in real-time mode with no output file of their own, instead handing their events straight to the Windows Event Log service, which formats and writes them into the familiar `Security.evtx`, `Application.evtx`, `System.evtx`, and the various channel-specific EVTX files under `%SystemRoot%\System32\winevt\Logs\`. And a session called `SgrmBroker`, short for System Guard Runtime Monitor, runs in buffering mode collecting platform integrity attestation data in support of Virtualization-Based Security, feeding its own dedicated broker process.

RtBackup: the shadow copy of what's still in memory

A folder easy to overlook, `%SystemRoot% \System32\LogFiles\WMI\RtBackup\`, holds ETW backing files that

support real-time sessions when consumers are absent or fall behind. Their presence and contents depend on the active sessions and system state. Files associated with event-log sessions may offer extra evidence, but they are not a guaranteed archive of events erased from Security.evtx. Check what survives on the system rather than assuming every circular buffer has a disk copy. These are ordinary `.etl` files, readable with `tracerpt.exe`, the Windows Performance Analyzer, or a tool like SilkETW, though access to the folder itself is restricted to SYSTEM, TrustedInstaller, and Administrators. Forensic triage frameworks like KAPE and Velociraptor both ship a dedicated artifact definition specifically for collecting this folder.

The rest of the standard autologger roster

Beyond the sessions already described, a stock Windows install runs a further handful worth knowing by name. `WiFiSession` captures Wi-Fi association and disassociation events, network profile changes, and BSSID information. `WdiContextLog` captures system state at the moment WDI diagnostics detect a problem. `ReadyBoot`, writing into `%SystemRoot%\Prefetch\ReadyBoot\`, captures disk I/O and driver load order during boot specifically, in support of boot-time optimization. `NetCore` captures network-stack events including connection establishment, DNS activity, and TCP resets. `AppModel` tracks UWP app launch, suspend, and termination. `UBPM`, the Unified Background Process Manager, logs background task trigger events. `LwtNetLog` captures lightweight network telemetry. And `SleepStudy` tracks power and battery-related events, including per-process battery drain and activity during Connected Standby.

The critical providers, organized by what they watch

Separately from any specific session, it's worth knowing the identity of the ETW providers that show up most often across investigations, since the same provider can be subscribed to by multiple sessions, or queried directly by a tool like Sysmon or a custom collector. Grouped by category, the providers that matter most are:

Process and thread execution: `Microsoft-Windows-Kernel-Process` emits process and thread start and stop events along with image load details, including the command line and token SID; `Microsoft-Windows-Kernel-General` covers system-level changes like the clock being adjusted; `Microsoft-Windows-ProcessStateManager` tracks the running, suspended, and frozen lifecycle states specific to UWP apps; `Microsoft-Windows-Security-Auditing` emits the underlying events behind the Security event log; and `Microsoft-Windows-Win32k` captures keyboard and mouse input, IME activity, and clipboard access at a low level.

File system activity: `Microsoft-Windows-Kernel-File` covers file creation, reads, writes, deletes, renames, and security changes, with fields that vary by event; file identifiers may need matching to name events; `Microsoft-Windows-StorPort` covers storage I/O requests and latency; `Microsoft-Windows-NTFS` covers NTFS-specific metadata operations and volume mount and dismount events; and `Microsoft-Windows-FilterManager` covers minifilter drivers attaching and detaching and their pre- and post-operation callbacks.

Networking: `Microsoft-Windows-TCPIP` and `Microsoft-Windows-Kernel-Network` both cover connection state and data transfer at slightly different layers, `Microsoft-Windows-NDIS` covers adapter-level packet events, `Microsoft-Windows-Winsock-AFD` covers

socket-level operations with full process context, `Microsoft-Windows-DNS-Client` covers query names, responses, and TTLs, `Microsoft-Windows-WinHttp` and `Microsoft-Windows-WebIO` cover HTTP and HTTPS request activity, `Microsoft-Windows-WinInet` covers the legacy internet stack, and `Microsoft-Windows-NetworkProfile` covers changes between public, private, and domain network categories.

Registry activity is covered in detail by the provider, `Microsoft-Windows-Kernel-Registry`, emitting create, open, close, query, set, and delete operations with the full key path involved.

Security and audit activity draws on `Microsoft-Windows-Security-Auditing` again, alongside `Microsoft-Windows-AppLocker` for application-control decisions, `Microsoft-Windows-Windows Defender` for antivirus scan and detection events, `Microsoft-Windows-CodeIntegrity` for signature verification results, and `Microsoft-Windows-Crypto-DPAPI` for the master-key access events behind the credential-protection system covered in part twelve.

And PowerShell activity is covered by `Microsoft-Windows-PowerShell` for the classic Windows PowerShell engine and a separate `PowerShellCore` provider for the cross-platform PowerShell 7 and later.

Between the specific autologger sessions in this chapter and the provider catalog just given, the picture that emerges is a system doing an enormous amount of structured, low-level self-observation before a user has even finished logging in, all of it running through the same tracing infrastructure, whether the eventual destination is a telemetry upload to Microsoft, a line in Event Viewer, or nothing more than a circular in-memory buffer waiting to be overwritten.

PART EIGHT

the scheduler's secrets

CHAPTER 25

The scheduled tasks Windows runs without asking

Every one of these tasks is stored as an XML file under `%SystemRoot%\System32\Tasks\`, organized into the same folder structure you'd see in the Task Scheduler management console, and a task can be marked hidden, meaning it can be omitted from the default console view; View > Show Hidden Tasks reveals it.

Compatibility and appraisal

A cluster of tasks under `\Microsoft\Windows\Application Experience\` exists specifically to support Windows' upgrade and compatibility decisions. The Microsoft Compatibility Appraiser runs `compattelrunner.exe` daily and after idle periods, performing a full assessment of hardware, driver, and application compatibility to inform upgrade eligibility, writing its findings directly into `Amcache.hve` (covered in chapter 4) and forwarding them into the telemetry pipeline; it runs as a hidden task. Alongside it, `PcaPatchDbTask` updates the Program Compatibility Assistant's database daily, `ProgramDataUpdater` syncs device inventory into Amcache's inventory tables daily, `StartupAppTask` scans everything configured to launch at logon about a minute after sign-in and reports startup-impact metrics, `AitAgent` runs weekly to report Application Impact Telemetry, a legacy cousin of the older Customer Experience Improvement Program, and `SdbinstMergeDbTask` merges custom application-compatibility shim databases at boot.

The Customer Experience Improvement Program itself, despite being largely superseded, still leaves tasks under `\Microsoft\Windows\Customer Experience Improvement Program\`: `Consolidator` runs `wsqmcons.exe` daily for CEIP-enrolled machines, gathering system configuration, hardware specs, installed applications, and usage patterns for submission to `sqm.microsoft.com`; `KernelCeipTask` gathers kernel-level metrics like boot timing and driver faults at every boot; `UsbCeip` fires on USB device insertion to log device usage telemetry including vendor and product IDs; and `BthSQM` runs on a schedule to log Bluetooth pairing and usage.

Disk, input, and system health

A handful of tasks exist purely to check the health of the machine itself. Under `\Microsoft\Windows\Autochk\`, a task called `Proxy` acts as a reporting bridge for CHKDSK errors, submitting disk-health reports through the WER pipeline. Under `\Microsoft\Windows\DiskDiagnostic\`, `Microsoft-Windows-DiskDiagnosticDataCollector` runs roughly every seven days during idle time to collect SMART attributes: reallocated and pending sector counts, uncorrectable errors, and the drive's model, serial number, and firmware version, and a companion `Microsoft-Windows-DiskDiagnosticResolver` analyzes that data afterward and can trigger a user-facing warning if failure looks likely. Under `\Microsoft\Windows\TextServicesFramework\`, `MsCtfMonitor` runs at every logon to track input method usage: keyboard layout switches and text-input statistics, relevant if handwriting or voice input is enabled. Under `\Microsoft\Windows\Speech\`, `SpeechModelDownloadTask` periodically pulls updated speech-recognition language models from a Microsoft CDN, implicitly logging which language packs are in use.

Feedback, diagnosis, and device inventory

Under `\Microsoft\Windows\Feedback\Siuf\`, `DmClient` periodically submits user feedback scenarios correlated with the device's telemetry ID, and once it completes, `DmClientOnScenarioDownload` fetches the next feedback scenario to present from `settings-win.data.microsoft.com`. Under `\Microsoft\Windows\Windows Error Reporting\`, `QueueReporting` triggers whenever the network becomes available after a crash, uploading anything sitting in the WER report queue described in chapter 15. Under `\Microsoft\Windows\Diagnosis\`, a task simply named `Scheduled` runs the built-in troubleshooter during a weekly maintenance window, checking for known issues and submitting a diagnostic report. Under `\Microsoft\Windows\DeviceInformation\`, `Device` runs at boot and again every twenty-four hours to build a full hardware census, covering CPU model, RAM, disk size and type, GPU, display resolution, touch capability, battery presence, BIOS version, TPM version, and Secure Boot state, all of it feeding directly into telemetry, while a companion `Device User` task runs at logon to capture per-user census data like account type, language, and region. And under `\Microsoft\Windows\PI\`, standing for Policy-based Instrumentation, a COM-triggered task called `Sqm-Tasks` periodically gathers Software Quality Metrics covering session-level data and Windows component feature usage.

Power, maintenance, gaming, updates, and the registry

A task under `\Microsoft\Windows\Power Efficiency Diagnostics\`, `AnalyzeSystem`, runs `powercfg.exe /energy` during a maintenance window to produce a full energy-efficiency report, including per-process CPU wake patterns and sleep-state blockers, saved as an

HTML file directly under `%SystemRoot%\System32\`. Under `\Microsoft\Windows\Maintenance\`, `WinSAT` runs the Windows System Assessment Tool after upgrades to benchmark CPU, disk, GPU, and memory performance, storing its results as XML under `%SystemRoot%\Performance\WinSAT\DataStore\`.

Gaming gets its own pair of tasks under `\Microsoft\XblGameSave\`: `XblGameSaveTask` uploads game save data to the cloud on game exit, and `XblGameSaveTaskLogon` syncs saves back down at logon, both associating activity with the account's Gamertag. Windows Update maintains several tasks of its own under `\Microsoft\Windows\WindowsUpdate\`: a periodic `Scheduled Start` task triggers an update scan and reports which KBs are installed or pending, `sih` runs daily to download and silently apply self-healing remediation scripts, `sihpostreboot` runs any follow-up actions after a restart, and `Automatic App Update` handles Microsoft Store app update scanning. Finally, under `\Microsoft\Windows\Registry\`, a task called `RegIdleBackup` used to back up registry hives to `%SystemRoot%\System32\config\RegBack\` every ten days, a genuinely useful forensic snapshot mechanism that Microsoft disabled by default starting with Windows 10 version 1803, meaning on most modern machines this backup simply doesn't happen anymore unless it's been manually re-enabled.

Taken together, this is dozens of Microsoft-authored tasks running silently across boot, logon, idle periods, and fixed schedules, with names, schedules, visibility, and availability varying by build and configuration, each one feeding a specific piece of the wider telemetry and diagnostics picture covered throughout this book.

CHAPTER 26

How the Task Scheduler keeps score

Behind every one of the tasks in the previous chapter, and every custom task a user or an attacker might create, sits the same underlying machinery: an XML file on disk, a registry cache tracking its runtime state, and an event log recording every time it fires.

The task XML itself

Every task's definition lives as UTF-16LE XML under `%SystemRoot% \System32\Tasks\`, shared by 32-bit and 64-bit tasks; legacy Task Scheduler 1.0 .job files use `%SystemRoot%\Tasks\.` Within the XML, a `RegistrationInfo` block records when the task was created, its author, and its full path within the scheduler's own namespace. A `Triggers` block can hold any combination of a boot trigger, a logon trigger tied to a specific user, a time trigger with an optional repetition interval, or an event trigger driven by an XPath query over Windows Event Log events. A `Principals` block specifies which account the task runs as, often by SID, along with its run level and logon type; a logon type of `S4U` (Service for User) is worth noting specifically, since it lets a task run under a specific account's context without needing that account's password stored anywhere. A `Settings` block can mark the task hidden, a common technique for both legitimate Microsoft tasks and attacker-created ones alike, and can set an execution time limit and a policy for what

happens if multiple instances try to run at once. Finally, an `Actions` block defines what the task actually does, most commonly an `Exec` action naming a command, its arguments, and a working directory, though a `ComHandler` action, invoking a COM object by class ID, is also possible and considerably less obvious to a casual reviewer.

The registry cache

Because reading and parsing XML files for every task on every scheduler query would be slow, Windows maintains a parallel cache of task metadata in the registry, and this cache is worth treating as one source for whether a task is enabled and when it last ran. It lives at `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\`. A `Tree` subkey mirrors the folder structure of task paths, with each task name pointing to a GUID that identifies its corresponding entry under a sibling `Tasks` subkey. There, each GUID-named key holds the task's ID, its full path, a version-dependent CRC32 or SHA-256 hash of the task's XML definition (useful for detecting whether the file has been modified since it was registered), binary-encoded copies of its actions and triggers, a security descriptor, and, most usefully, a `DynamicInfo` value holding the task's actual runtime state: its creation time, the FILETIME of its last run, the FILETIME of its last successful completion, when that field is present, and the HRESULT result code from that last run, all packed into a compact binary structure at fixed offsets. Four further subkeys, Logon, Boot, Plain, and Maintenance, index tasks by trigger type and persistence behavior, letting the scheduler quickly find, for instance, every task that needs to fire at the next user logon without scanning the entire cache.

The event log

When enabled, the task history channel can record lifecycle events in `Microsoft-Windows-TaskScheduler/Operational.evtx`. Event ID 106 marks a task being registered, whether newly created or updated, and includes its full path; 140 marks an update to an existing task; 141 marks deletion, again including the task's name and the SID responsible; 200 marks an action actually starting, including the process ID it launched; 201 marks that action completing, with its result code; 102 marks successful completion of the task instance; 202 indicates an action failure; 203 marks an action that failed to start in the first place; event ID 322 means a new instance was ignored because the task was already running, while 325 records an instance queued to launch immediately. Between the XML definition, the registry cache's dynamic runtime state, and this event log, a task's entire lifecycle, who created it, what it does, when it last ran, and whether it succeeded, is reconstructable even if the task itself has since been deleted, as long as at least one of these three sources hasn't also been cleaned up.

PART NINE

the network and the cloud

CHAPTER 27

The telemetry pipeline's journey to Microsoft

Chapter 24 traced the DiagTrack pipeline as far as a compressed package sitting in `%ProgramData%\Microsoft\Diagnosis\Packages\`, waiting to be sent. This chapter picks up from there, at the network boundary itself.

The primary destination for that data is the Universal Telemetry Client's main endpoint, `v10.events.data.microsoft.com`, reached over ordinary HTTPS, carrying compressed JSON-like event bundles tagged with a device ID, session ID, hardware fingerprint, and whatever feature-usage events have accumulated since the last upload. Older builds, and certain fallback conditions, route through a legacy pair of endpoints instead, `vortex.data.microsoft.com` and `v10.vortex-win.data.microsoft.com`. A separate endpoint, `settings-win.data.microsoft.com`, is primarily a configuration endpoint the device pulls from, downloading DiagTrack's own configuration: which event categories are currently enabled and at what sampling rate. Crash and error data takes an entirely different path, through Windows Error Reporting's Watson infrastructure at `watson.telemetry.microsoft.com` and `umwatson.telemetry.microsoft.com`, with a related endpoint, `oca.telemetry.microsoft.com`, handling Online Crash Analysis correlation across similar crash reports from other machines. And explicit user feedback submissions go to `feedback.microsoft.com` directly.

Put simply, the local pipeline described in chapter 24, ETW events flowing into an autologger, staged by DiagTrack, compressed into a package, queued for upload, terminates at one of a small handful of well-known Microsoft domains, almost always over TLS on port 443, with the specific domain depending on whether what's being sent is routine telemetry, a settings check-in, a crash report, or direct user feedback.

CHAPTER 28

Windows Update, Delivery Optimization, and identity sync

Keeping Windows itself updated

The Windows Update client, running as the `wuauserv` service, talks to a family of Microsoft domains under `*.windowsupdate.microsoft.com`, `*.update.microsoft.com`, and `sls.update.microsoft.com`, exchanging the list of installed KB updates, a hardware ID used for driver matching, and basic language and version information. A separate service, Delivery Optimization (`DoSvc`), handles the peer-to-peer side of update distribution, exchanging update chunks directly with other nearby machines on the same network or across the broader internet in addition to Microsoft's own CDN endpoints (`*.do.dsp.mp.microsoft.com` and `*.emdl.ws.microsoft.com`), logging bytes sent and received, peer IP addresses, and CDN health in local ETL logs readable with Get-DeliveryOptimizationLog. A third component, the Update Session Orchestrator (`UsoSvc`), coordinates the overall scan, download, and install sequence and writes a scan history to `%ProgramData%\USOShared\Logs\`. Microsoft Store app updates go through a related but separate service, `InstallService`, checking against `displaycatalog.mp.microsoft.com` and `storeedgefd.dsx.mp.microsoft.com` for update manifests and licensing.

Identity and settings sync

A separate cluster handles account identity. The Microsoft Account Sign-in Assistant (`wlidsvc`, introduced in chapter 2) refreshes account tokens against `login.live.com`; organizational sign-in uses separate identity components and `login.microsoftonline.com`. Azure AD device join, handled through `dsregcmd`, registers the device's join state and TPM-based hardware attestation against `enterpriseregistration.windows.net` and the same `login.microsoftonline.com` endpoint. And Settings Sync, running through `SettingSyncHost.exe`, pushes a defined set of settings categories up to `settings.live.net` and `ds.settings.live.net`: theme, Wi-Fi passwords (only if the user has opted in), browser passwords (also opt-in), language, ease-of-access preferences, and Start menu layout. Exactly which categories sync is controlled by a set of individually toggleable registry values under `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\SettingSync\Groups\`, one `Enabled` value per category: accessibility settings, browser settings, credentials, language, package theming, desktop personalization, and Start layout can each be switched independently, governed overall by a master `SyncPolicy` value one level up.

CHAPTER 29

OneDrive and the Connected Devices Platform

OneDrive's local footprint

OneDrive ships installed by default on Windows 10 and 11 and runs not as a service but as an ordinary user process, `%LocalAppData%\Microsoft\OneDrive\OneDrive.exe`, launched automatically through `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\OneDrive`. Its state is spread across a fairly elaborate folder structure: a version-specific subfolder holding `FileSyncClient.exe`, a `logs\` folder holding a proprietary binary telemetry cache called `UserTelemetryCache.otc` alongside separate plain-text `SyncDiagnostics.log` files for personal and business accounts, a `settings\` folder holding per-account `.ini` files (named after the account's Consumer ID, or CID) that track folder GUIDs and sync state, plus a downloaded `ClientPolicy.ini`, and, elsewhere in the profile, a log-transfer-service folder and, under `%ProgramData%`, setup and update logs.

The sync state can live in `SyncEngineDatabase.db`, whose tables vary by release. Versions parsed by OneDriveExplorer use `od_ClientFile_Records`, `od_ClientFolder_Records`, and `od_ScopeInfo_Records` for file, folder, and scope metadata. They can hold names, parent/resource IDs, sizes, status, ETags, timestamps, and local hashes. Check the actual tables: hash algorithms and hydration details vary, and the database need not hold a complete history. Transfer and deletion

information may be available in logs and state that vary by version. OneDriveExplorer also parses `SafeDelete.db` tables such as `items_moved_to_recycle_bin` and `filter_delete_info.` These can preserve deletion evidence, though a local database need not record every file ever synced. Retained hashes and metadata can still be useful after a local file is removed.

The plain-text `SyncDiagnostics.log` complements the database with a running, timestamped narrative: entries recording a file's local path and cloud ID as it's uploaded, confirmation once an upload completes with its resulting ETag, conflict detection when local and server versions diverge, and, notably, explicit `DeleteItem` entries that record a cloud-side deletion even if both the local file and the cloud recycle bin have since been emptied. A separate set of registry values under `HKCU\SOFTWARE\Microsoft\OneDrive\` and its `Accounts\Personal\` and `Accounts\Business1\` subkeys record the account's client ID, email address, display name, consumer ID, local root folder, and last sign-in time, with business accounts additionally recording their SharePoint tenant URL and last sync timestamp.

One more detail is worth understanding on its own: OneDrive's "online-only" files, the ones showing a cloud icon rather than being fully downloaded, are implemented as genuine NTFS reparse points, carrying file attribute flags that trigger a download the moment the file is opened or its data accessed, tagged with a dedicated reparse point type (a slightly different tag distinguishes a personal OneDrive file from a OneDrive for Business one). The practical forensic effect is that such a file appears completely normal in a directory listing and in the MFT, but its $DATA can be unallocated; a separate $`REPARSE_POINT` holds the reparse buffer, meaning the file's

cloud content identifier can be recovered even when the actual file content has never once been downloaded to the machine being examined.

Office and Microsoft 365's own telemetry

Separately from OneDrive's file-sync telemetry, the Office application suite maintains its own diagnostic data collection, running independently of DiagTrack and governed by its own privacy controls inside each application (under File, Account, and the account privacy settings in modern Office builds). What it gathers follows the same general shape as the rest of this book's telemetry chapters: which features and commands get used inside Word, Excel, Outlook, and the rest of the suite, add-in load and crash activity, and performance and error data specific to the Office applications themselves, staged locally before upload on a schedule independent of the operating system's own telemetry pipeline. For an investigation touching document activity specifically, this Office-level telemetry is worth treating as a separate evidence source from anything covered elsewhere in this book, since disabling Windows' own diagnostic data collection has no effect on it, and vice versa.

The Connected Devices Platform, joined up

Chapter 18 covered the local databases behind Windows Timeline, `ActivitiesCache.db` and `cdp.db`, in detail. What's worth adding here, from the network and service side, is the retention picture: Timeline activity itself is retained in the cloud for roughly thirty days once synced, while the local database also expires activity. Cloud clipboard supports text below 100 KB; local history supports items up to four

megabytes each; history can retain multiple entries. Nearby Share's device-discovery broadcasts, covered briefly in chapter 18 as well, remain session-only by design and leave nothing behind once the session ends.

CHAPTER 30

Push notifications, SmartScreen, MAPS, and cloud clipboard

Push notifications

The Windows Push Notification Service, or WNS, is the delivery mechanism for cloud-sent toast and background notifications, run through the system-level `WpnService` and its per-user counterpart `WpnUserService`. It maintains a persistent TLS 1.2 connection out to `*.notify.windows.com`, and its state, including an app-and-user-specific channel URI that expires after thirty days and is renewed for cloud notifications, is tracked under `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\PushNotifications\`. Chapter 20 already covered where the resulting notification content actually lands, in `wpndatabase.db` 's `Notification` and `NotificationHandler` tables; this is simply the network-facing half of the same system, and WNSPushChannel stores the channel URI, linked to the notification handler.

SmartScreen and MAPS

Two related but distinct systems handle reputation checking. SmartScreen's file-reputation check queries `checkappexec.microsoft.com` and `nis.microsoft.com` with a SHA-256 hash of the file in question, its PE metadata, the URL it was downloaded from, and some process context; its separate URL-reputation check queries `browser.smartscreen.microsoft.com`

with URL and reputation information, rather than only a partial hash of the URL. Running alongside SmartScreen, the Microsoft Active Protection Service (MAPS) is Defender's own cloud lookup system, querying `*.wdcp.microsoft.com` and a fallback `wdcpalt.microsoft.com` with file hashes, sizes, and PE metadata, and, in cases where Microsoft's cloud engine flags a file as needing closer inspection (subject to cloud-protection and sample-submission settings), submitting the full file sample. MAPS operates at two levels, controlled by a registry value, `HKLM\SOFTWARE\Microsoft\Windows Defender\Spynet\SpynetReporting`: on Windows 10 and later, values of either 1 or 2 select Advanced membership; the older Basic distinction no longer applies; a companion value, `SubmitSamplesConsent`, governs that submission behavior more precisely.

Cloud clipboard, briefly revisited

Chapter 20 covered cloud clipboard's local storage format and sync path in full. The one detail worth adding here is the plain three-state summary of its behavior: with cloud sync enabled, text below 100 KB can sync; local history supports twenty-five items of up to four megabytes each; with only local history enabled, the same kind of data is retained on the machine but never leaves it; and with the feature disabled outright, nothing persists past the ordinary single-item clipboard at all.

Knowing which network you're on

A related but separate concern is how Windows figures out what kind of network it's connected to in the first place, handled by

NCSI, hosted by `NlaSvc` on Windows 10 and the Network List Service (netprofm) on Windows 11. It determines internet reachability by probing a fixed URL, `http:// www.msftconnecttest.com/connecttest.txt`, and separately confirms connectivity through a DNS probe against `dns.msftncsi.com`, which is expected to resolve to a specific, hardcoded address; both probe targets are configurable through values under `HKLM\SYSTEM\CurrentControlSet\Services\NlaSvc\Parameters\Internet\`. Whatever network it identifies gets recorded persistently: `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\NetworkList\Profiles\{GUID}\` stores the network's name, its category (public, private, or domain), and when it was first and most recently connected, while two related keys, `Signatures\Managed\` and `Signatures\Unmanaged\`, hold lower-level network signatures used to recognize a previously seen network automatically. This same registry structure comes up again in the next chapter, from the Wi-Fi side specifically.

CHAPTER 31

Wi-Fi profiles, Credential Manager, and token stores

Every network you've ever joined

Every Wi-Fi network a machine has connected to leaves behind an XML profile at `%ProgramData% \Microsoft\Wlansvc\Profiles\Interfaces\{InterfaceGUID}\{ProfileGUID}.xml`. Inside, the profile records the network's name both as plain text and as hex-encoded SSID bytes, whether the network broadcasts its SSID, its connection type (infrastructure or ad-hoc), whether it auto-connects, and, under a security block, the authentication and encryption scheme in use (WPA2-Personal with AES encryption being typical) along with the actual key material itself, DPAPI-encrypted with the machine's own key, which is why an administrator on the same machine can retrieve the plaintext password with a tool that calls `CryptUnprotectData`, while offline decryption requires the appropriate `DPAPI` system keys as well as the XML. The command-line tool `netsh wlan export profile`, run as an administrator, is the standard, fully supported way to pull these out.

A parallel registry record complements the XML profiles. The same `NetworkList\Profiles\{GUID}\` key introduced in the previous chapter stores a Wi-Fi network's category and its first-and last-connected timestamps as 16-byte `SYSTEMTIME` values, while `NetworkList\Signatures\Unmanaged\{SigGUID}\` links back to that profile and separately records the network's name, the six-byte MAC address of the gateway router seen at first connection, and the DNS suffix observed at that time. That gateway MAC address is

worth calling out specifically: it ties a machine's presence to a particular physical router, which means it can help establish which physical location a laptop was actually sitting in in a way a network name alone (which could be a common SSID reused at many locations) cannot.

A related, purely local-network artifact rounds this out: current and recent IP configuration for each network adapter, held under `HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces\{InterfaceGUID}\`, recording assigned and DHCP-issued IP addresses, subnet mask, default gateway, DNS servers, and, importantly, a `LeaseObtainedTime` value marking exactly when a DHCP lease was acquired, which together with the lease's expiry gives a fairly precise window for when a machine was present on a particular network.

Where Windows keeps your saved passwords

Windows Credential Manager and the newer Vault system are where saved website, network share, and application passwords actually live. The older format stores one file per credential under `%AppData%\Microsoft\Credentials\`, each DPAPI-encrypted; the newer Vault format organizes credentials under `%LocalAppData%\Microsoft\Vault\{VaultGUID}\`, with a `Policy.vpol` file defining the vault's own encryption policy and individual `.vcrd` files holding each encrypted credential record. A parallel system-wide vault exists under `%ProgramData%\Microsoft\Vault\`, and SYSTEM-context credentials, the kind a service account might use, live under the system profile's own credentials folder. A legacy credential blob has a binary header recording a version number, a credential type (generic, domain password, domain certificate, or a domain-visible password), some flags, and a `LastWritten`

FILETIME marking the most recent modification, ahead of the actual DPAPI-encrypted payload. The supported way to enumerate what's stored, without ever seeing the actual passwords, is the built-in `cmdkey /list` command or the more detailed `vaultcmd /listcreds:` for either the Windows Credentials or Web Credentials store specifically. A much older, separate mechanism is worth a mention too: older Internet Explorer versions stored autofill form passwords under `HKCU\SOFTWARE\Microsoft\Internet Explorer\IntelliForms\Storage2\`, keyed by a SHA-1 hash of the relevant URL, still DPAPI-encrypted.

Chapter 37 returns to DPAPI itself, the encryption system underlying essentially every credential store in this section, and walks through exactly how its master keys are structured and, from a forensic standpoint, how they can be decrypted offline.

Beyond saved passwords specifically, a separate cache, `%LocalAppData%\Microsoft\TokenBroker\Cache\`, holds the OAuth2 access and refresh tokens that UWP apps use to authenticate against a Microsoft account or an Azure AD identity, acquired and refreshed automatically by the Token Broker service introduced in chapter 1.

CHAPTER 32

Xbox, Edge sync, the Microsoft Store, and the enterprise endpoint list

Xbox and gaming

Gaming on Windows runs through its own small service cluster: `XblAuthManager` for Xbox Live authentication, `XblGameSave` for cloud save synchronization, `XboxNetApiSvc`, and `xbgm` for Xbox Game Monitoring, talking to a family of endpoints under `*.xboxlive.com` and related domains for authentication and game-clip metadata, with local state tracked under `HKLM\SOFTWARE\Microsoft\XboxLive\` and its per-user counterpart, and package-specific data under `%LocalAppData%\Packages\Microsoft.XboxGamingOverlay_*\LocalState\`. What actually gets uploaded includes recorded gameplay clips captured through Game Bar (flowing to OneDrive or Xbox's own cloud storage), timestamped achievement unlocks, per-title session duration, party join and leave events (though not voice content itself), and, tying it all together, the account's Gamertag.

Edge sync, briefly revisited

Chapter 19 covered Edge's local footprint and its account-level sync categories in full detail, including the specific SQLite schemas behind its history and downloads. The short version, for reference alongside the rest of this part's endpoint list: Edge syncs favorites, history, passwords, open tabs, autofill data, the installed extension list, and general settings through `sync.microsoft.com`, with separate telemetry flowing to `data.microsoft.com`.

The Microsoft Store

Store functionality runs through `InstallService`, `ClipSVC`, and `LicenseManager`, with the Store's own UI processes being `WinStore.App.exe` on older builds and `MicrosoftStore.exe` on newer ones. These talk to `displaycatalog.mp.microsoft.com` for catalog data, `licensing.mp.microsoft.com` for license validation, `pti.store.microsoft.com`, and `storesdk.dsx.mp.microsoft.com`, with local state split between a package-specific `LocalState` folder and the shared `%ProgramData%\Microsoft\Windows\AppRepository\`used elsewhere for UWP app inventory. What gets transmitted includes app purchase and install events with timestamps, license validation checks tying a device ID and account ID to a specific app entitlement, and periodic update pings reporting the installed version, the available version, and telemetry opt-in status.

The full endpoint picture

Pulling every domain mentioned across this part, and a few from earlier chapters, into one place, the following is the enterprise-relevant summary of Microsoft endpoints a Windows machine routinely contacts, along with the port each one typically uses:

Category	Endpoint pattern	Port
DiagTrack telemetry	`v10.events.data.microsoft.com`	443
DiagTrack legacy	`vortex.data.microsoft.com`	443
DiagTrack settings	`settings-win.data.microsoft.com`	443

Category	Endpoint pattern	Port
WER crash reports	`watson.telemetry.microsoft.com`	443
Online Crash Analysis	`oca.telemetry.microsoft.com`	443
Defender MAPS	`*.wdcp.microsoft.com`	443
SmartScreen	`checkappexec.microsoft.com`	443
Windows Update	`*.windowsupdate.microsoft.com`	443 / 80
Delivery Optimization	`*.do.dsp.mp.microsoft.com`	443 + 7680 (peer-to-peer)
WNS push	`*.notify.windows.com`	443
Microsoft account auth	`login.live.com`	443
Azure AD auth	`login.microsoftonline.com`	443
OneDrive	`*.files.1drv.com`, `onedrive.live.com`	443
Activity Feed / Timeline	`activityfeed.microsoft.com`	443
Settings Sync	`settings.live.net`	443
Xbox Live	`*.xboxlive.com`	443
Cortana / Bing	`*.bing.com`, `*.cortana.ai`	443
NLA connectivity probe	`www.msftconnecttest.com`	80
Time sync	`time.windows.com`	123 (NTP)

Category	Endpoint pattern	Port
Certificate revocation	`crl.microsoft.com`	80
Maps	`*.virtualearth.net`	443
Microsoft Store	`displaycatalog.mp.microsoft.com`	443

For a network defender, this list doubles as a baseline: traffic to these domains, from a stock Windows machine, is ordinary background noise rather than a sign of anything unusual, which is exactly why understanding it matters. It's the traffic that doesn't fit this pattern that's worth a second look.

PART TEN

persistence and the early boot

CHAPTER 33

BITS, BootExecute, MountedDevices, and the ordinary autostart locations

BITS as more than a download manager

The Background Intelligent Transfer Service, introduced in chapter 1 as an ordinary networking component, maintains its job queue in a database, `qmgr.db`, under `%ProgramData% \Microsoft\Network\Downloader\`, built in ESE format on current Windows versions (older systems used `qmgr0.dat` and `qmgr1.dat`). Every BITS job recorded there carries a job ID, a display name, a priority level, a type (upload or download), a current state, the SID of the owning user, and a list of files involved, each with its remote URL, local destination path, size, and transfer progress. What makes BITS interesting from a persistence standpoint is a job property called `NotifyCmdLine`, which lets whoever created the job specify a command to run automatically when that job completes or, just as usefully for an attacker, when it errors out; combined with `NotifyFlags` controlling exactly which condition triggers it, this turns an ordinary background download job into a delayed or conditional code-execution trigger that doesn't need a scheduled task or a registry Run key at all. Existing jobs, for any user, can be enumerated with `bitsadmin /list /allusers /verbose` or the PowerShell `Get-BitsTransfer` cmdlet, and the `Bits-Client/Operational` event log channel introduced in chapter 14 records the job lifecycle directly: event ID 3 for job creation, 4 for a job completing, 59 for a transfer starting, 60 for transfer stop, and 61 for a transfer error, any of which is worth cross-referencing against a job's `NotifyCmdLine` value.

Code that runs before Windows really starts

A registry value at `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\BootExecute`, defaulting to the single entry `autocheck autochk *` (the command that triggers a disk check when one's been scheduled), is processed by the Session Manager Subsystem, `smss.exe`, at a point in the boot sequence earlier than almost anything else in this book: before the Win32 subsystem itself has finished initializing, and therefore before any user-mode security product, antivirus included, has had the chance to start. Anything added here has to be a native executable, built against `ntdll.dll` directly rather than the ordinary Win32 API surface most software relies on, which is a real barrier to casual abuse but not an impossible one, and it's exactly this timing advantage, running before ordinary user-mode defenses, though kernel protections such as ELAM initialize earlier, that makes an entry here so valuable to an attacker able to place one.

A cluster of closely related registry values live in the same key and are worth knowing about together. `SetupExecute` serves a similar early-execution role specifically during Windows Setup. `PendingFileRenameOperations` is a legitimate mechanism installers and Windows Update itself use to replace a file that's currently locked, by scheduling the replacement to happen at the next boot before the file is reopened by anything else, a mechanism malware has also been observed abusing to swap in a malicious file at the earliest possible opportunity. And an `Execute` value, alongside an initial `S0` command slot, rounds out the small set of hooks available at this stage of startup.

Every drive that's ever been plugged in

A single registry key, `HKLM\SYSTEM\MountedDevices`, quietly stores persistent volume identities and drive-letter mappings, which can outlast connections but do not form a complete assignment history. Each value is named either after a drive letter, in the form `\DosDevices\C:`, or after a volume GUID, in the form `\??\Volume{GUID}`, and its binary data encodes the underlying volume's identity: for an ordinary MBR-partitioned disk, that data is a compact twelve bytes, combining the four-byte disk signature read from the disk's master boot record with an eight-byte value giving the starting byte offset of the specific partition on that disk. Because volume IDs can persist, even long after the corresponding physical device has been disconnected and put away, it becomes one of the more durable records of external media history on a machine, and it pairs naturally with two further sources: the `USBSTOR` registry key, which independently records every USB storage device's vendor, product, and serial number, and the `Setupapi.dev.log` (or, on older systems, `Setupapi.log`) device-installation log, which records the exact date and time a given device was first installed, pinning down precisely when a drive was connected rather than just confirming that it once was.

The ordinary autostart locations

Set against the exotic timing tricks of BootExecute and the fileless elegance of a WMI subscription, the most common autostart mechanism on Windows is almost mundane by comparison, which is exactly why it's worth covering here rather than skipping it as too obvious to mention. A small set of registry keys, most commonly `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` for every user on the machine and the equivalent

`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` for a single account, list a plain command line under an arbitrary value name for each program that should launch automatically at logon. A sibling pair of `RunOnce` keys works the same way but normally deletes entries before execution; `!` defers deletion until afterward, originally meant for post-install or post-update setup steps that should only ever fire a single time. Older, largely superseded variants of the same idea, `RunServices` and `RunServicesOnce`, still occasionally turn up on systems with a long upgrade history.

Alongside the registry, a purely file-based equivalent has existed since long before Windows had much of a registry to speak of: the Startup folder, at `%AppData%\Microsoft\Windows\Start Menu\Programs\Startup` for a single user and `%ProgramData%\Microsoft\Windows\Start Menu\Programs\StartUp` for every user on the machine, where simply placing a shortcut or executable is enough to have it launch at the next logon. Because both mechanisms, the Run keys and the Startup folder, are so old, so simple, and so heavily monitored by essentially every antivirus and EDR product as a matter of course, they've become somewhat less attractive to a sophisticated attacker than the techniques covered elsewhere in this part, but they remain by a wide margin the most common autostart location for ordinary legitimate software, and any review of what launches automatically on a given machine still has to start here before moving on to the subtler mechanisms this book covers elsewhere.

Volume shadow copies as a window into the past

One further mechanism belongs in this part precisely because it works against every technique covered so far in this book, including deliberate attempts to erase evidence: the Volume

Shadow Copy Service, or VSS, which Windows uses to take periodic, point-in-time snapshots of an entire NTFS volume, primarily in support of System Restore and File History but available for far broader use. Each snapshot preserves the file system largely as it existed at the moment it was taken, which means a file that's since been deleted, modified, or even timestomped on the live volume can often still be recovered, completely intact, from an older shadow copy. Existing snapshots can be listed with `vssadmin list shadows`, and a specific snapshot can be mounted directly as its own read-only volume with `mklink /d` pointed at its device path, making its entire historical file system browsable like any ordinary drive. Because a shadow copy is a snapshot of the whole volume rather than of any single artifact, it's also one of the few ways to retrieve a clean, undisturbed copy of a locked file like `Amcache.hve`, `SRUDB.dat`, or a live SYSTEM hive without needing to stop the service that owns it, a technique this book has referenced repeatedly in earlier chapters. Its limitation is retention: shadow copies are capped by configured disk space and age out over time, so how far back they reach depends entirely on how the machine has been configured and how much disk churn has happened since.

CHAPTER 34

WMI and its persistence tricks

Windows Management Instrumentation is, at its foundation, a management and scripting interface, the mechanism behind everything from `Get-WmiObject` queries to enterprise configuration tools. Its full object database, called the WMI repository, lives at `%SystemRoot%\System32\wbem\Repository\`, split across a main data file (`OBJECTS.DATA`), a B-tree index (`INDEX.BTR`), and a set of three rotating mapping files, `MAPPING1.MAP` through `MAPPING3.MAP`, that WMI cycles through as a form of transactional safety, so that a crash mid-write doesn't corrupt the whole repository. The repository is organized into namespaces, with `root\cimv2` holding most general system and hardware data, `root\default` and `root\security` covering their respective areas, and, most relevant to this chapter, `root\subscription`, which is where permanent event consumers live.

A permanent WMI event subscription is one of the more elegant persistence techniques available on Windows, precisely because it requires no file on disk at all beyond the repository itself, no scheduled task, and no registry Run key. It's built from three linked objects. An `__EventFilter` defines a trigger condition as a WQL query, something like watching for any new process being created, evaluated on a defined polling interval. An `__EventConsumer`, of which several types exist, defines what happens when that filter matches: a `CommandLineEventConsumer` runs an arbitrary command line, an `ActiveScriptEventConsumer` executes embedded VBScript or JScript directly, and further consumer types can write to a log file, send an email through SMTP, or write an entry to the NT event log. A third object, `__FilterToConsumerBinding`, is what actually

links a specific filter to a specific consumer, and it's only once this binding exists that the subscription becomes live.

Because none of this shows up in Task Scheduler, in a Run key, or as a service, hunting for it means querying the repository directly: `Get-WmiObject -Namespace root\subscription -Class __EventFilter`, and the equivalent queries against `__EventConsumer` and `__FilterToConsumerBinding`, will enumerate whatever's currently registered, and both Sysinternals Autoruns (in its dedicated WMI tab) and third-party tools built specifically around this repository format, such as python-cim and PyWMIPersistenceFinder, can parse the repository files offline without needing a live, bootable copy of the machine at all. The `WMI-Activity/Operational` event log channel, introduced in chapter 14, is worth a second look here specifically: event ID 5857 marks a WMI provider starting and 5858 records a failed `WMI` client operation, both fairly routine, but event IDs 5860 and 5861, marking temporary event subscriptions and permanent filter-to-consumer bindings respectively, are two of the highest-value single event IDs anywhere in this book for catching this exact persistence technique at the moment it's established, rather than having to go looking for it after the fact.

PART ELEVEN

scripts, audits, and what gets logged

CHAPTER 35

PowerShell is always watching itself

PowerShell has become both an administrator's primary tool and an attacker's preferred scripting environment, and largely in response to the latter, Microsoft built PowerShell's logging into four genuinely independent layers, each configured separately and each capturing a different slice of what actually happened.

Module logging

Module logging, controlled by `HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging`, records the details of pipeline execution for whichever modules are named in its `ModuleNames` list (commonly configured with a wildcard to cover everything). Enabled, it writes event ID 4103 to the Microsoft-Windows-PowerShell/Operational log, capturing the parameter values passed into cmdlets as they run.

Script block logging

Script block logging, at `HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging`, is generally considered the single most valuable of the four layers, because it logs the actual content of every script block PowerShell parses and runs, written to `Microsoft-Windows-PowerShell/Operational.evtx` as event ID 4104. If a script block is long enough to exceed a single event's practical size, PowerShell splits it across multiple 4104 events, each carrying a message number and a total count so they can be reassembled in order, all tagged with a shared

`ScriptBlockId` GUID that ties the fragments together. A companion setting, invocation logging, adds event IDs 4105 and 4106 marking the start and stop of a given block's execution. What makes this layer so effective against evasion specifically is timing: `PowerShell` can log decoded `-EncodedCommand` text and script blocks created during execution. Some obfuscated expressions still need analysis; the log does not always show their final result.

Transcription

Transcription, configured at `HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription`, works differently from the other three layers in that it writes ordinary, human-readable plain text files rather than structured event log entries, named `PowerShell_transcript.{hostname}. {random}.{timestamp}.txt` and saved to a configured output directory. Its distinguishing feature is that it's the only one of the four that captures both sides of a session, the commands typed or run and their actual output and results as they appeared on screen, with an optional invocation header recording timestamps for individual commands.

AMSI

The Antimalware Scan Interface sits apart from the other three, because it isn't a logging mechanism so much as a content-inspection one that happens to leave a log trail as a side effect. A component called `amsi.dll` loads into PowerShell and other scripting hosts, and before a script actually executes, it calls

`AmsiScanBuffer()`, handing the script's content to whatever antivirus engine is registered to receive it, giving that engine a chance to inspect and potentially block genuinely malicious content even if it was never written to disk. AMSI activity emits its own ETW provider, `Microsoft-Antimalware-Scan-Interface`, and when Defender specifically flags AMSI-submitted content, it logs event ID 1116 for the detection and 1117 for whatever action was taken, both to the `Microsoft-Windows-Windows Defender/Operational` channel. It's worth knowing that AMSI has a well-documented history of in-memory bypass techniques, generally involving patching the `AmsiScanBuffer` function directly in the loaded DLL to make it always report clean content, which is one reason investigators treat AMSI detections as useful but not as a complete picture on their own.

The command history nobody thinks to check

One further artifact, unrelated to any of these four logging layers and unaffected by whether any of them are even enabled, is `PSReadLine`'s own history file, sitting at `%AppData% \Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`. It's an ordinary plain-text file recording, line by line, interactive commands retained by `PSReadLine`, subject to history settings and sensitive-data filtering, persisting across sessions rather than resetting each time a new console window opens, which makes it one of the simplest and most complete records of interactive command-line activity available on the whole machine, entirely independent of whether an administrator ever turned on any of the formal audit policies above. It's worth noting this specifically covers the interactive console host; PowerShell ISE and integrated terminals in editors like VS Code may or may not feed the same history file, depending on configuration.

CHAPTER 36

The DNS cache, security auditing, and AppLocker

The DNS cache

Windows keeps a resolver cache purely in memory, by design, with no file backing it at all, which means it has to be captured live: running `ipconfig /displaydns` while the `Dnscache` service is still up dumps every currently cached record, showing the queried name, record type, remaining time-to-live, and the resolved data itself. The cache's overall size and maximum entry lifetime are configurable through `HKLM\SYSTEM\CurrentControlSet\Services\Dnscache\Parameters\`, under `CacheHashTableSize` and `MaxCacheEntryTtlLimit` respectively. Its forensic value is specific and worth stating plainly: a cached entry may reflect a query or a Hosts-file preload, which is meaningfully different from proving a connection actually succeeded, so it can surface an attacker's command-and-control domain even when the corresponding network connection itself was blocked by a firewall before it ever completed. Because none of this survives a reboot or even a cache flush, it has to be part of live triage rather than something recoverable after the fact, unless it happens to be preserved inside a memory image captured while the cache was still populated.

Security auditing

The Local Security Authority maintains its own master audit policy in a binary structure, `PolAdtEv`, sitting inside the `HKLM\SECURITY`

hive, a hive so restricted that reading it directly requires SYSTEM-level access. In practice, both administrators and investigators interact with this policy through friendlier interfaces: `auditpol.exe` for querying and setting the modern, granular per-subcategory policy, or the Local Security Policy console for much the same thing through a GUI. Nine broad audit categories exist, Account Logon, Account Management, Detailed Tracking, DS Access, Logon and Logoff, Object Access, Policy Change, Privilege Use, and System, each with its own set of finer-grained subcategories that can be independently enabled.

Whatever the policy captures lands in `Security.evtx`, and a fairly small set of event IDs account for the large majority of what an investigator actually looks for. Under logon and logoff, event ID 4624 marks a successful logon and carries a logon-type field worth memorizing on its own: type 2 is an interactive console logon, 3 is a network logon (an SMB connection, for instance), 4 is a batch logon typically associated with a scheduled task, 5 is a service logon, 7 is a workstation unlock, 8 is NetworkCleartext (not necessarily plaintext on the wire), 9 is a logon using alternate credentials (`RunAs` with `/ netonly`), 10 is a remote interactive logon over RDP, and 11 is a cached-credential logon used when a domain controller isn't reachable. Event ID 4625 marks a failed logon attempt, with an accompanying sub-status code narrowing down why. Event ID 4634 (and its more specific counterpart 4647) marks a logoff, 4648 marks a logon performed with explicitly supplied alternate credentials, the signature of a `RunAs` operation, and 4672 marks a logon being granted special, administrator-level privileges.

Under detailed tracking, event ID 4688 marks process creation and, when a separate Group Policy setting for command-line auditing is also enabled, includes the full command line the process was launched with, alongside its parent process ID and a token

elevation type describing whether UAC elevation was involved; 4689 marks the corresponding process termination. Under account management, 4720 marks a new user account being created, 4722 marks one being enabled, 4724 marks a password-reset attempt, and a family of group-membership events, 4732 for a security-enabled local group, 4728 for a global group, and 4756 for a universal group, marks an account being added to each respective group type. Under object access, event ID 5140 marks a network share being accessed and 5145 provides a more detailed, per-file breakdown of that same share access, while 4656 and 4663 mark an object handle being requested and actually used respectively, though both require the specific object in question to have its own audit-relevant access control entry configured before they'll fire at all. Under policy change, event ID 4719 marks the audit policy itself being altered, a strong tampering indicator on its own, and, arguably the single most important event ID in this entire chapter, event ID 1102 marks the security audit log being cleared outright, an event generated by normal Security-log clearing regardless of audit policy. Offline deletion or other tampering may leave no such event. Finally, under privilege use, event IDs 4673 and 4674 mark the use of a sensitive privilege, the kind normally reserved for system-level operations.

AppLocker

Where SmartScreen and Defender make reputation-based decisions, AppLocker enforces an explicit allow list, and its policy is stored under `HKLM\SOFTWARE\Policies\Microsoft\Windows\SrpV2\`, broken into separate rule collections for executables, MSI installers, scripts, DLLs, and packaged UWP apps, each collection holding a set of GUID-identified rules. A rule can match on three different bases: a publisher rule matches against the Authenticode

signing certificate of the file, a path rule matches a specific file system location, and a hash rule matches an exact SHA-256 hash of the file itself, the most restrictive but also the most brittle of the three, since it breaks the moment the file is updated. AppLocker's decisions land in their own dedicated event log channels: the EXE and DLL channel logs event ID 8003 when a file would have been blocked under enforcement but was allowed because the policy is only running in audit mode, and 8004 when a file is actually blocked outright, with a parallel pair, 8006 and 8007, covering MSI installers and scripts, and a further set of channels covering packaged app deployment and execution specifically. Whether any of this happens at all, in an observing-only capacity or as genuine enforcement, comes down to a single setting per rule collection: not configured, audit only, or enforce, and the distinction matters enormously when reviewing AppLocker's logs, since different event IDs and messages identify allowed executions and those that would have been blocked in audit mode.

PART TWELVE

credentials, identity, and Defender

CHAPTER 37

Windows Hello, DPAPI, and the credential vaults

Windows Hello and the NGC credential store

Windows Hello, the PIN, fingerprint, and face-recognition sign-in system, is built on a framework Microsoft calls NGC, for Next Generation Credentials. Its registry structure lives at `HKLM\SOFTWARE\Microsoft\Cryptography\Ngc\`, organized into per-user, GUID-named container keys under the relevant account SID, each one referencing a named key container along with a key ID and a signing-hash algorithm. The actual private key material itself is kept, wherever the hardware supports it, inside the machine's TPM, specifically so that it's non-exportable: there is no file anywhere on disk containing the raw key, because the key was generated inside the TPM and never leaves it. Where a TPM isn't available, a software-based key storage provider substitutes, with key files under `%SystemRoot%\ServiceProfiles\LocalService\AppData\Roaming\Microsoft\Crypto\Keys\` in the Windows 10 builds studied in 2021, considerably weaker than TPM protection but still not a simple flat file. Chapter 2 already covered the separate biometric template store, `%SystemRoot% \System32\WinBioDatabase\`, used for the fingerprint and face data itself; NGC is the layer above that, tying whichever biometric or PIN factor succeeds to the actual cryptographic key used for sign-in.

The PIN specifically is worth understanding correctly, because it's commonly and incorrectly treated as equivalent to a password. It isn't: a Hello PIN never leaves the device and is used only

to unlock a key that's already bound to that specific physical device, which means a captured PIN is useless on any other machine, unlike a password, which works anywhere the corresponding account can authenticate. Under the hood, this PIN-unlocked key is typically used to protect a further, DPAPI-secured symmetric key, which in turn can unlock a user's ordinary DPAPI master key, described next, meaning Windows Hello effectively provides an alternate, hardware-bound path into the same encrypted-secrets system a password would otherwise unlock.

DPAPI: the master key behind almost everything

The Data Protection API, DPAPI, referenced repeatedly across this book, whether protecting Wi-Fi passwords, saved credentials, application secrets, or a browser's stored logins, ultimately uses user or `SYSTEM` master keys. Understanding where those keys live and how they're derived explains why so many seemingly unrelated artifacts share the same underlying protection.

Each user's DPAPI master keys are stored as small binary files under `%AppData%\Microsoft\Protect\{UserSID}\`, one file per master key GUID, and a machine-context equivalent lives under `%SystemRoot%\System32\Microsoft\Protect\S-1-5-18\` for SYSTEM-protected secrets. A file inside that folder called `Preferred` simply records the GUID of whichever master key is currently the active one. The master key itself is stored encrypted, and decrypting it depends on the account and key-file format. A password-derived credential and the user's `SID` help unlock user keys; the file records the derivation settings and cipher. Modern files commonly use AES; older formats differ. Once decrypted, that master key is what actually protects every individual DPAPI blob scattered across the

artifacts covered in earlier chapters, from a Wi-Fi profile's stored password to a Credential Manager vault entry to other DPAPI-protected application secrets.

Because a user's password can change over time, and because DPAPI needs old secrets to remain decryptable after a password change, a second file, `CREDHIST`, sitting alongside the master key files, maintains a chained history of previous password-derived hashes, letting the system walk backward through past passwords as needed to decrypt a master key that was originally protected under an older one. For domain-joined machines, a further recovery path exists entirely independent of the user's password: a domain-wide DPAPI backup key, held on the domain controller, can decrypt domain-user master keys containing matching domain-backup recovery data, a legitimate administrative recovery mechanism that's also, if a domain controller is ever compromised, a way to retroactively decrypt DPAPI-protected secrets across every machine in that domain. In practice, offline DPAPI decryption is usually performed with purpose-built tools: Mimikatz's DPAPI module, the more forensics-oriented SharpDPAPI, or Impacket's `dpapi.py` script, all of which implement this same derivation chain to recover a plaintext master key given either the user's password or an extracted domain backup key.

CHAPTER 38

Defender's detection history, certificate stores, and the firewall log

What Defender caught, and where it put it

When Windows Defender quarantines a file, it doesn't delete it outright; it moves it, encrypted, into `%ProgramData% \Microsoft\ Windows Defender\Quarantine\`, split into an `Entries\`folder holding detection metadata and a `ResourceData\` folder holding the file's actual (encrypted) content, each one named after a SHA-1-derived identifier rather than its original filename. The "encryption" involved is Defender's own long-documented scheme, RC4 encryption using a fixed key, which means quarantined samples can be reliably decrypted and recovered for further analysis with any of several publicly available extraction scripts built around that same fixed key. Beyond quarantine itself, Defender keeps detailed operational logs under a `Support\` folder in the same location: `MpCmdRun.log` records command-line scan invocations, while a series of `MPLog-*.log` files hold a running record of scan history, individual detections, and signature-definition updates. A registry key at `HKLM\SOFTWARE\Microsoft\Windows Defender\Scan\` tracks overall scan scheduling and history, while a sibling `Threats\` key holds one entry per detected threat, recording its threat ID, the exact timestamp of detection, the action taken (quarantined, removed, or allowed), and the file's path at the moment it was flagged. The 1116/1117 events covered in chapter 35 are general Defender detection/action events, also used for AMSI detections; older event sets include 1006 and event ID 5001 for real-time protection

being disabled, the latter worth flagging on its own as a common precursor to whatever comes next in an intrusion.

Certificate stores

Windows maintains certificate stores at both the per-user and machine level, and both are worth checking in most investigations, since they record more than just ordinary web browsing trust decisions. The per-user store sits in the registry under `HKCU\SOFTWARE\Microsoft\SystemCertificates\{StoreName}\Certificates\`, with each certificate keyed by its own thumbprint, split across named stores: `My` for personal certificates (used for things like client authentication and email signing), `Root` for trusted root certificate authorities, `CA` for intermediate authorities, `TrustedPublisher` for code-signing certificates explicitly trusted, `Disallowed` for certificates explicitly blocked, and `AuthRoot` for the Microsoft-managed list of trusted roots. A parallel structure exists under `HKLM` for machine-wide stores, and a file-based mirror of the personal store lives at `%AppData%\Microsoft\SystemCertificates\My\Certificates\`. The forensic value of these stores runs in a couple of directions: the `Root` store's contents can reveal whether a corporate proxy or, less innocently, an adversary-in-the-middle tool has injected its own certificate authority to intercept otherwise encrypted TLS traffic, since either would need to add a trusted root certificate to make that interception invisible to the browser, while the `My` store's history can establish exactly when a given client certificate was available for authentication.

The firewall log

Windows Firewall's packet-level logging is genuinely useful and, by default, entirely disabled. Where it's turned on, whether through `netsh advfirewall` or a Group Policy value under `HKLM\SOFTWARE\Policies\Microsoft\WindowsFirewall\`, the resulting log writes to `%SystemRoot%\System32\LogFiles\Firewall\pfirewall.log` in the W3C extended log file format, recording, for the dropped packets and successful connections selected for logging, the date and time, whether it was allowed or dropped, the protocol involved, source and destination IP addresses and ports, the packet size, TCP flag and sequence information, ICMP type and code where relevant, and whether the packet was being sent or received. Configuration values control the log's path, its maximum size, and separately whether dropped packets, successful connections, or both get recorded; because the log rotates by a fixed maximum size rather than by date, a very busy firewall can end up with a much shorter effective retention window than an idle one, so its absence of older entries doesn't necessarily mean nothing happened, only that the log filled up and moved on. This file-based log naturally complements the single Security event log entry that fires per blocked connection, event ID 5157, introduced back in chapter 1 as one of the ways Defender's own firewall service documents what it's stopped.

PART THIRTEEN

the last mile

CHAPTER 39

Installer footprints, the services registry, and Group Policy artifacts

What an MSI installer leaves behind

Software installed through the Windows Installer leaves a surprisingly durable trail. A hidden system folder, `%SystemRoot% \Installer\`, caches local copies of installed .msi packages, often stripped of embedded cabinets and renamed to internal filenames, alongside any `.msp` patch files applied afterward. The installed product's metadata lives in two closely related registry locations: `HKLM\SOFTWARE\Classes\Installer\Products\{ProductCode}\` tracks the Windows Installer's own view of the product, while the far more commonly referenced `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\{GUID}\` (the key that populates the ordinary "Installed apps" list) records the display name, version, publisher, installation date (which patches or repairs can change), install location, an uninstall command string, and an estimated installed size.

Verbose installer logging, when enabled through `HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\Logging`, uses a compact letter-code scheme to select exactly what gets recorded: `v` for verbose output, `o` for out-of-disk-space conditions, `i` for status messages, `c` for initial UI parameters, `e` for all error messages, `w` for non-fatal warnings, `a` for the startup of each action, `r` for action-specific records, `m` for out-of-memory conditions, `u` for user requests, `p` for terminal properties, with `+` and `!` as modifiers to append to an existing log file and flush after every line respectively. When active, these logs

land as `%Temp%\MSI*.log` files, one per installation attempt, and installer activity separately reaches `Application.evtx` under the `MsiInstaller` source, with event ID 1033 marking a successful install and a range of numbered error events covering specific failure conditions.

The services registry as a persistence map

Every Windows service, driver-based or not, has an entry under `HKLM\SYSTEM\CurrentControlSet\Services\{ServiceName}\`, and reading that key's values carefully turns it into one of the more direct persistence maps on the whole system. A `Type` value distinguishes a service running in its own dedicated process (0x10) from one sharing a process with others (0x20), and separately identifies kernel-mode and file-system drivers (0x1 and 0x2 respectively). A `Start` value determines when the service activates: 0 for boot-start (loaded by the kernel loader itself, earliest of all), 1 for system-start (loaded early by the I/O manager), 2 for automatic start once the service control manager comes up, 3 for demand start, including service triggers, and 4 for disabled outright. An `ErrorControl` value determines how the system reacts if the service fails to start. An `ImagePath` value points at the actual binary, and for a service running under a specific account rather than the default LocalSystem context, an `ObjectName` value records which one. `DisplayName` and `Description` provide the human-readable labels, `DependOnService` lists any services that must start first, and a `FailureActions` binary value defines what happens automatically if the service crashes, including whether it should restart itself. For services that run inside a shared `svchost.exe` process rather than their own, the actual DLL doing the work is named in a `ServiceDLL` value nested under a `Parameters` subkey, exactly the pattern seen repeatedly throughout part one of this book.

The forensic utility here is direct: comparing the full list of services configured with `Start=2` against a known-good baseline for the same Windows build is one of the more reliable ways to spot a newly persisted service that shouldn't be there, and the `Type` value's driver flags specifically flag anything installed as a kernel-mode driver, a category of persistence considerably harder to detect from user mode than an ordinary service ever is.

Group Policy's local and domain artifacts

Locally applied Group Policy settings live under `%SystemRoot% \System32\GroupPolicy\Machine\` and the equivalent `User\` folder, with the actual registry-based settings stored in a binary file called `registry.pol`, opening with the signature `PReg` and encoding each configured setting as a sequence of key path, value name, and value data triples. For domain-joined machines, a synchronized cache of every applicable Group Policy Object lives under `%SystemRoot%\System32\GroupPolicy\DataStore\`, with a parallel `GroupPolicyUsers\` folder holding the per-user equivalent. The command `gpresult /h` produces a full HTML report of the actual, effective, resultant policy currently applied to a machine, accounting for every GPO in the chain. Group Policy processing itself is logged to the `Microsoft-Windows-GroupPolicy/Operational` channel introduced in chapter 14: 4000/4001 mark computer/user policy processing starting, and 8000/8001 mark successful completion; failures use other event IDs. Finally, `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Group Policy\History\{GPO-GUID}\` maintains a local record of exactly which GPOs have been applied to the machine and when, broken down further by which specific client-side extension (registry settings, software installation, scripts, and so on) processed each one.

CHAPTER 40

The EVTX format itself

Chapter 14 toured the Windows Event Log system channel by channel. This closing chapter of part thirteen looks at the file format underneath every one of those `.evtx` files, since understanding its structure is what makes recovery possible even after a log has been cleared or a file has been partially corrupted.

Every EVTX file opens with a fixed header, identified by the eight-byte signature `ElfFile\0`, followed by the number of the first and current chunk in the file, the ID that will be assigned to the next record written, the header's own size, a minor and major version number, the size of the whole header block, a count of chunks in the file, and a CRC32 checksum protecting the header itself. The body of the file is divided into chunks, each 64 KiB, with the chunk count depending on log size, and each chunk carries its own signature, `ElfChnk\0`, its own range of record numbers, an offset marking where free space begins within it, and the actual event records themselves.

Each individual event record inside a chunk opens with its own four-byte signature, a size field, a monotonically increasing record ID unique within the channel, a FILETIME timestamp, and then the event's actual content, encoded not as plain-text XML but as Binary XML, a tokenized, templated encoding designed to avoid repeating the same tag names and structure for every single event; a chunk builds up a small library of templates as it fills, and subsequent records reference those templates rather than re-encoding the same structure from scratch. The record closes with its size value repeated a second time at the very end, a detail that exists

specifically to let a parser walk backward through a chunk's records as easily as forward.

How a channel behaves once it fills up is governed by a maximum log size (`MaxSize` under `HKLM\SYSTEM\CurrentControlSet\Services\EventLog\{LogName}\` for classic logs), alongside a retention policy that, on the overwhelming majority of modern Windows configurations, means the log simply overwrites its oldest chunks in a circular fashion once it's full, rather than growing indefinitely or refusing new entries.

The reason all of this technical detail earns a chapter on its own is recovery. Because each chunk carries its own signature and its own internal checksum, a chunk doesn't strictly need an intact file header, or even an intact surrounding file, to be independently parsed and validated. That means that even after an attacker has cleared `Security.evtx` outright (an act that, as chapter 36 covered, normally generates event ID 1102), fragments of the cleared chunks frequently remain recoverable by carving unallocated disk space for the `ElfChnk` signature directly, entirely independent of whatever the live, current file now contains. The standard tools for this kind of work are `EvtxECmd`, again from Zimmermann's suite, the open-source `python-evtx` library for programmatic parsing, and PowerShell's own `Get-WinEvent` cmdlet, which can open a standalone or carved `.evtx` file directly through its `-Path` parameter without needing it registered as an active channel at all.

PART FOURTEEN

making sense of it all

CHAPTER 41

Tools of the trade

Across the previous forty chapters, a fairly consistent toolkit keeps reappearing, and it's worth gathering in one place, organized by what you're actually trying to look at rather than scattered across the chapters where each tool first came up.

A single suite accounts for a surprising fraction of it. Eric Zimmermann's tools, commonly referred to collectively as EZ Tools, cover an enormous share of the artifacts in this book with a consistent command-line interface and CSV or JSON output: `PECmd` for Prefetch (chapter 4), `AppCompatCacheParser` for ShimCache and `AmcacheParser` for Amcache (also chapter 4), `SrumECmd` for the SRUM database (chapter 5), `RECmd` for general registry parsing including UserAssist (chapter 8), `ShellBagsExplorer` for Shellbags (chapter 10), `LECmd` for individual LNK files and `JLECmd` for jump lists (chapter 11), `MFTECmd` for the Master File Table and the USN change journal together (part four), and `EvtxECmd` for Windows Event Log files (chapters 14 and 40). Between this one family of tools and a small number of specialized companions, most of the artifacts in this book can be parsed without needing a full commercial forensic suite at all.

Outside that suite, a handful of tools are worth knowing by name for the gaps it doesn't cover. `LogFileParser`, from Joakim Schicht, handles NTFS's `$LogFile` transaction log (chapter 13), a format specialized enough that it hasn't been folded into the more general-purpose tools. `srum-dump` is a well-established open-source alternative specifically for SRUM. Volatility3 covers memory forensics broadly, including its dedicated `windows.callbacks`

plugin for enumerating kernel notification routines (chapter 21) and hibernation-layer or conversion support, which depends on version (chapter 16), with Arsenal Recon's commercial Hibernation Recon tool covering similar ground. For live ETW work, `tracerpt.exe` ships with Windows itself, Microsoft's Windows Performance Analyzer provides a much richer graphical view of the same `.etl` files, and SilkETW offers a lighter-weight, scriptable alternative for real-time provider subscription (chapter 24). For WMI's repository specifically, `python-cim` and `PyWMIPersistenceFinder` parse the repository files offline without needing the machine booted at all (chapter 34). And for DPAPI decryption, Mimikatz's DPAPI module, the more forensics-oriented SharpDPAPI, and Impacket's `dpapi.py` script all implement the same master-key derivation chain described in chapter 37.

A different category of tooling sits above all of this: full disk imaging tools like FTK Imager, used to create a forensically sound copy of a drive before any of the artifact-specific parsers above ever touch it, and complete forensic suites like Autopsy, X-Ways Forensics, or EnCase, which wrap many of these same parsing capabilities into a single case-management environment. None of the tools named in this book are a substitute for sound acquisition and chain-of-custody practice; they're simply what turns a raw file or registry hive into something a human can actually read.

CHAPTER 42

Making sense of timestamps

Four different timestamp formats recur constantly throughout this book, and mixing them up is one of the easiest ways to build an inaccurate timeline. This chapter collects all four in one place.

FILETIME, the format behind the overwhelming majority of timestamps in this book, from Prefetch run times to BAM entries to `$STANDARD_INFORMATION`, is a 64-bit count of 100-nanosecond intervals since midnight, January 1, 1601, UTC. To convert one to an ordinary Unix timestamp, divide by 10,000,000 to get seconds since that 1601 epoch, then subtract 11,644,473,600, the number of seconds between 1601 and 1970, to land on the familiar Unix epoch.

Chrome and Edge time, used throughout the `History` and `downloads` tables covered in chapter 19, shares the exact same 1601 epoch as FILETIME but counts in microseconds rather than 100-nanosecond ticks. Converting it means dividing by 1,000,000 instead of 10,000,000 before subtracting that same 11,644,473,600-second offset.

Unix time, the simplest of the four, is just a count of ordinary seconds since midnight, January 1, 1970, UTC, with no further conversion needed. It shows up in this book in Windows Recall's `WindowCapture` table (chapter 7) and in the `InstallDate` value recording when Windows itself was installed (chapter 16).

MS-DOS date and time, the oldest format covered here, packs a date into 16 bits and a time into a separate 16 bits, used inside Shellbag folder entries (chapter 10) and inside ZIP archive metadata generally. The date half splits into a 7-bit year offset from 1980, a 4-bit month, and a 5-bit day; the time half splits into a 5-bit hour, a

6-bit minute, and a 5-bit value representing seconds divided by two, since two seconds is the finest resolution the format can represent.

One habit is worth adopting regardless of which format you're converting: always work in UTC until the very last step of presenting a timeline to a person. Most parsing tools will happily convert straight to local time for readability, which is convenient right up until you're correlating artifacts from a machine that's changed time zones, or comparing timestamps across two machines configured differently, at which point a silent local-time conversion is exactly the kind of thing that quietly introduces an hour or more of error into an otherwise precise timeline.

CHAPTER 43

Anti-forensics: what defeats each artifact, and what survives

Every chapter so far has described an artifact as though it simply persists once created. In practice, a motivated user, whether an ordinary person trying to protect their privacy or an attacker trying to cover their tracks, has plenty of techniques available to remove, alter, or avoid leaving many of them. What's worth understanding, cutting across this entire book, is that almost none of these techniques defeat everything at once, because the artifacts in this book don't all depend on each other.

Timestomping, rewriting a file's `$STANDARD_INFORMATION` timestamps directly (chapter 12), is effective against the timestamps most casual tools display by default. But the `$FILE_NAME` attribute's own timestamps, set by the kernel rather than through the documented API timestomping tools use, usually survive untouched, and the mismatch between the two is itself a strong tell. The `$UsnJrnl` change journal (chapter 13) separately records the metadata-change event with its own accurate timestamp regardless of what the file's displayed timestamps now claim, and if the file had already been indexed, thumbnailed, or prefetched before the tampering occurred, those earlier artifacts retain their original, honest timing.

Secure deletion, overwriting a file's content before removing it, can defeat recovery from overwritten blocks, but not other copies or remapped SSD blocks. It does nothing, however, to the `$UsnJrnl` record of the file having been created, renamed, and deleted in the first place, nor to any Shellbag, jump list, LNK file, or

RecentDocs entry that referenced it while it existed (part three), nor to Prefetch, ShimCache, or Amcache entries if the file was ever executed (chapter 4), nor to a thumbnail cache entry if it was ever previewed in Explorer (chapter 17).

Clearing the Security event log normally triggers event ID 1102 when cleared through the running Event Log service (chapter 36), and the RtBackup folder's shadow ETL files can independently retain data that predates the clear (chapter 24). Clearing one channel also does nothing to any of the others: System, Application, Sysmon if it's deployed, and every feature-specific channel covered in chapter 14 remain untouched, and none of SRUM, BAM, or UserAssist have any dependency on the event log at all.

Living off the land, using only tools already present on the system to avoid dropping new malicious files, does not avoid execution artifacts. A LOLBin still triggers the kernel's process and image-load notification routines (chapter 21) regardless of how it was invoked, still shows up in Prefetch, ShimCache, and Amcache if it genuinely ran, and if it's PowerShell-based, enabled script block logging can capture decoded or dynamically executed script content, without guaranteeing complete deobfuscation (chapter 35).

Disabling `DiagTrack` stops its uploads, while other components can still contact Microsoft, but it doesn't retroactively delete telemetry already uploaded, and it does nothing to the large majority of artifacts in this book, which are purely local byproducts of the file system, the registry, and kernel-level tracing that keep functioning regardless of DiagTrack's state.

Fileless or memory-only execution defeats disk-based artifacts specifically: no LNK file, no Prefetch entry, no `$MFT` record for a payload that genuinely never touches disk. But it does nothing to

hide the process once it actually runs in memory, which still fires the same kernel notification routines and ETW events as any other process (part seven), can still be recovered from a hibernation file, page file, or live memory capture (chapter 16), and, if the persistence mechanism itself is a WMI permanent event subscription rather than a scheduled task or Run key, that subscription is stored directly in the file-backed `WMI` repository regardless of how fileless the payload it triggers might be (chapter 34).

The pattern that runs through every one of these examples is the same one this book has returned to throughout: no single artifact tells the whole story, and no single anti-forensic technique defeats every category of artifact at once, because the categories were never built by a single team with a single, defeatable design. One closing observation worth sitting with: the anti-forensic tool itself, whatever it is, still has to run to do its work, and a program that runs leaves the same execution evidence covered in part two as any other program. Often enough, the clearest single sign that someone tried to cover their tracks is the record of them opening the tool they used to try.

CHAPTER 44

The privacy control registry: turning it off

Having spent this whole book describing what Windows collects and remembers, it's worth closing with the practical counterpart: the specific settings that actually turn each category off, organized to match the chapters where the underlying artifact was introduced.

The single broadest control is the overall telemetry level, set through `HKLM\SOFTWARE\Policies\Microsoft\Windows\DataCollection\AllowTelemetry`: 0 means off on Enterprise/Education/Server; 1 is Required (Basic), 3 Optional (Full); 2 (Enhanced) is only for Windows 10 1809 or earlier and Server 2016/2019, and this single value governs how much of the DiagTrack pipeline described across chapters 1, 24, and 27 is actually active. The DiagTrack service itself can be stopped and disabled outright, using `sc.exe` stop `DiagTrack`, then `sc.exe` config `DiagTrack` start= disabled (or `PowerShell Stop-Service` and `Set-Service`); startup configuration alone does not stop a running service or other data flows.

A number of narrower controls turn off individual features covered in this book. Windows Error Reporting's `Disabled` value under `HKLM\SOFTWARE\Microsoft\Windows\Windows Error Reporting\`, covered fully in chapter 15, turns off crash reporting specifically. Cortana and web-integrated search results are controlled by `AllowCortana` and `ConnectedSearchUseWeb` under `HKLM\SOFTWARE\Policies\Microsoft\Windows\Windows Search\`. Activity History, the feature behind the `ActivitiesCache.db`

database covered in chapter 18, is controlled by a trio of values under `HKLM\SOFTWARE\Policies\Microsoft\Windows\System\`: `EnableActivityFeed` for the feature broadly, and `PublishUserActivities` and `UploadUserActivities` for whether activity gets shared to the device at all and whether it syncs to the cloud, respectively. The advertising ID used to personalize ads across apps is controlled by `HKLM\SOFTWARE\Policies\Microsoft\Windows\AdvertisingInfo\DisabledByGroupPolicy` at the machine level. `TailoredExperiencesWithDiagnosticDataEnabled` separately controls per-user diagnostic-data personalization of tips and suggestions. Location services, behind the artifacts described in chapter 2, are controlled machine-wide by `DisableLocation` under `HKLM\SOFTWARE\Policies\Microsoft\Windows\LocationAndSensors\`.

Several features covered in detail earlier in this book have their own dedicated toggles already introduced in context, worth summarizing here for completeness: cloud clipboard sync and local history, covered in chapter 20, come down to `EnableClipboardHistory` and `SyncClipboardToCloud` for a single user, or `AllowClipboardHistory` and `AllowCrossDeviceClipboard` as machine-wide Group Policy equivalents. Windows Recall, covered in chapter 7, is governed by `HKCU\SOFTWARE\Policies\Microsoft\Windows\WindowsAI\DisableAIDataAnalysis=1`, with a corresponding Group Policy setting to disable snapshot saving machine-wide. SmartScreen and MAPS cloud-reputation checking, covered in chapter 30, come down to the `SpynetReporting` value for MAPS (1 and 2 both select Advanced on Windows 10 and later). SmartScreen has separate policies, including `EnableSmartScreen` under `HKLM\SOFTWARE\Policies\Microsoft\Windows\System` and `SmartScreenEnabled` under Microsoft Edge's policy key.

Two smaller, easy-to-miss controls round this out. Handwriting and typing personalization, which feeds custom dictionary and autocorrect suggestions back into Microsoft's language models, is governed by values under `HKCU\SOFTWARE\Microsoft\InputPersonalization\`. And for anyone working from an older Windows 10 installation with a long upgrade history, it's worth knowing that Wi-Fi Sense, a much earlier feature that let contacts share Wi-Fi network access automatically, was retired outright in later Windows 10 releases, so a machine that still shows related registry remnants isn't necessarily still exposed by that particular mechanism, just carrying old configuration state that no longer does anything.

Most of these settings do not erase existing local caches or data already uploaded to `Microsoft.` Recall's turn-off-snapshots policy also deletes saved snapshots. These controls affect particular forms of collection or upload; their scope depends on the setting and Windows version. That's the distinction this book has drawn from its opening chapter onward: between the local artifact, which persists as a byproduct of how Windows works, and the telemetry upload, which is a separate, deliberate choice that these settings govern. Turning a service off tomorrow does not erase what it already wrote today, and by now, forty-three chapters in, you know exactly where to look to check.

References

These sources were consulted for the technical accuracy review. Windows developments introduced after June 2026 are outside this book's scope. Studies of particular builds should be read within that scope. The companion change log records the exact passages, historical source versions, and unresolved questions.

1. 1102(S) The audit log was cleared. - Windows 10
learn.microsoft.com · source version/date: 2021-09

2. 4624(S) An account was successfully logged on. - Windows 10
learn.microsoft.com · source version/date: 2021-09

3. 4657(S) A registry value was modified. - Windows 10
learn.microsoft.com · source version/date: 2021-09

4. 4688(S) A new process has been created. - Windows 10
learn.microsoft.com · source version/date: 2022-01

5. __fastfail
learn.microsoft.com · source version/date: 2026-06-30

6. _IMAGE_INFO (ntddk.h) - Windows drivers
learn.microsoft.com · source version/date: 2026-05-29

7. About Event Tracing - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

8. About TraceLogging - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

9. about_Logging_Windows - PowerShell
learn.microsoft.com · source version/date: 2026-06-30

10. about_PSReadLine_Release_Notes - PowerShell
learn.microsoft.com · source version/date: 2026-06-30

11. Accessing UMDF Metadata in WER Reports - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

References

12. ADMX_WindowsExplorer Policy CSP
learn.microsoft.com · source version/date: 2025-03

13. Allocated Filter Altitudes - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

14. ANSSI: Analysis of Amcache
cyber.gouv.fr · source version/date: version recorded in change log

15. Arsenal Recon: Hibernation Recon
arsenalrecon.com · source version/date: version recorded in change log

16. Attacker Use Background Intelligent Transfer Service (BITS) | Google Cloud Blog
cloud.google.com · source version/date: version recorded in change log

17. Automatic Memory Dump - Windows drivers
learn.microsoft.com · source version/date: 2018-05

18. BAM internals
dfir.ru · source version/date: 2020-04

19. Barnaby Skeggs: Touch Screen Lexicon Forensics (WaitList.dat)
forensicfocus.com · source version/date: 2016-11

20. Being productive at work with Windows 7
blogs.windows.com · source version/date: 2011-04

21. BSI/ERNW SiSyPHuS Work Package 4: Telemetry
bsi.bund.de · source version/date: 2018

22. Change Journal Records - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

23. Chromium 137: favicon service implementation
github.com · source version/date: 2025

24. Chromium 137: time types and epochs
github.com · source version/date: 2025

25. Chromium 137: visit database implementation
github.com · source version/date: 2025

References

26. Cloud Sync Engines - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

27. CLR Exception E0434352
learn.microsoft.com · source version/date: 2021-11

28. Collecting User-Mode Dumps - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

29. Configure Windows diagnostic data in your organization - Windows Privacy
learn.microsoft.com · source version/date: 2025-09

30. Configuring and Starting an AutoLogger Session - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

31. Connection endpoints for Windows 11 Enterprise - Windows Privacy
learn.microsoft.com · source version/date: 2026-06

32. Continue user activity, even across devices - UWP applications
learn.microsoft.com · source version/date: 2021-05

33. CrowdStrike: Windows FeatureUsage taskbar forensics
crowdstrike.com · source version/date: version recorded in change log

34. Debugging Indirect Display Drivers - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

35. Decoding the parameters of a thrown C++ exception (0xE06D7363) - The Old New Thing
devblogs.microsoft.com · source version/date: 2010-07

36. Defender Policy CSP
learn.microsoft.com · source version/date: 2026-06

37. Desktop Activity Moderator - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

38. DeviceEvents table in the advanced hunting schema - Microsoft Defender XDR
learn.microsoft.com · source version/date: 2025-03

References

39. Digging into the System Resource Usage Monitor (SRUM)
velociraptor-docs.org · source version/date: 2019-12

40. Digital Forensics - Prefetch Artifacts
countuponsecurity.com · source version/date: 2016-05

41. Dissect: Chromium browser profile parser
github.com · source version/date: 2026-06-10

42. Dissect: Defender quarantine format and decryption
github.com · source version/date: 2026-06

43. Dissect: DPAPI key discovery and decryption
github.com · source version/date: 2026-06-10

44. Dissect: DPAPI master-key formats
github.com · source version/date: 2026-06-10

45. Dissect: Windows ActivitiesCache parser
github.com · source version/date: 2026-06-10

46. Dissect: Windows MRU parsers
github.com · source version/date: 2026-06-10

47. Dissect: Windows notification database parser
github.com · source version/date: 2026-06-10

48. Dissect: Windows Recycle Bin parser
github.com · source version/date: 2026-07-01

49. Dissect: Windows Search ESE and SQLite parser
github.com · source version/date: 2026-06-10

50. Dissect: Windows Shellbags parser
github.com · source version/date: 2026-06-10

51. Dissect: Windows thumbnail-cache parser
github.com · source version/date: 2026-06-10

52. Domingues, Andrade, Frade: A Digital Forensic View of Windows 10 Notifications
pdfs.semanticscholar.org · source version/date: 2022-01-31

References

53. DosDateTimeToFileTime function (winbase.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

54. Downloads and the Mark-of-the-Web
textslashplain.com · source version/date: 2016-04

55. DPAPI backup keys on Active Directory domain controllers - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

56. Ejin Kim and Hyoung-Kee Choi: Security Analysis and Bypass User Authentication Bound to Device of Windows Hello in the Wild
onlinelibrary.wiley.com · source version/date: 2021-07-23

57. ELAM Driver Requirements - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

58. Enable debug logging for Netlogon service - Windows Client
learn.microsoft.com · source version/date: 2026-06-30

59. Enable insecure guest logons in SMB2 and SMB3 for Windows client and Windows Server
learn.microsoft.com · source version/date: 2026-06-30

60. Eric Zimmerman: Jump lists format
github.com · source version/date: 2015-01

61. EVENT_DESCRIPTOR (evntprov.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

62. Eventlog Key - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

63. EVT_CHANNEL_CONFIG_PROPERTY_ID (winevt.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

64. File Times - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

65. FileIo_ReadWrite class - Win32 apps
learn.microsoft.com · source version/date: 2018-05

References

66. Find out what's new in Windows and Office in October
blogs.windows.com · source version/date: 2018-10

67. FSCTL_GET_NTFS_FILE_RECORD - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

68. FSCTL_QUERY_USN_JOURNAL - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

69. FSCTL_READ_USN_JOURNAL - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

70. G DATA: Windows registry analysis of scheduled tasks
cyber.wtf · source version/date: version recorded in change log

71. Get Started with User Access Logging
learn.microsoft.com · source version/date: 2026-06-30

72. Get-DeliveryOptimizationLog (DeliveryOptimization)
learn.microsoft.com · source version/date: 2026-06-24

73. Guidance on configuring system services - Windows IoT Enterprise
learn.microsoft.com · source version/date: 2026-06-25

74. Handling Notifications - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

75. HKLM\SYSTEM\CurrentControlSet\Services Registry Tree - Windows drivers
learn.microsoft.com · source version/date: 2024-09

76. IAttachmentExecute (shobjidl_core.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

77. Image copy · Issue #1887 · volatilityfoundation/volatility3
github.com · source version/date: version recorded in change log

78. ipconfig
learn.microsoft.com · source version/date: 2026-06-30

79. Is Group Policy Slowing Me Down? | Microsoft Community Hub
techcommunity.microsoft.com · source version/date: version recorded in change log

References

80. JumpList: application ID mappings
github.com · source version/date: 2026-02

81. Kernel Live Dump Code Reference - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

82. libyal: System Resource Usage Monitor (SRUM) format
github.com · source version/date: 2021-06

83. libyal: Windows Explorer Thumbnail Cache database format
github.com · source version/date: 2024-05

84. libyal: Windows Prefetch file format
github.com · source version/date: 2026-06

85. libyal: Windows Search database formats
github.com · source version/date: 2021-06-05

86. libyal: Windows Shell Item format
github.com · source version/date: 2026-06

87. libyal: Windows Shortcut File (LNK) format
github.com · source version/date: 2023-09

88. libyal: Windows XML Event Log (EVTX) format
github.com · source version/date: 2024-09

89. Load Order Groups and Altitudes for Minifilter Drivers - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

90. Logging Mode Constants - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

91. Manage connections from Windows 10 and Windows 11 Server/Enterprise editions operating system components to Microsoft services - Windows Privacy
learn.microsoft.com · source version/date: 2026-02

92. Manage Privacy: SmartScreen Filter and Resulting Internet Communication
learn.microsoft.com · source version/date: 2016-08

References

93. Manage Recall for Windows clients
learn.microsoft.com · source version/date: 2025-12

94. Mark Baggett: srum-dump configuration and table mappings
github.com · source version/date: 2025-04

95. Master File Table (Developer Notes) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

96. Master File Table (Local File Systems) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

97. Microsoft BITS Client event manifest (Windows 10 build 17134)
github.com · source version/date: 2020-03

98. Microsoft Defender Antivirus event IDs and error codes - Microsoft Defender for Endpoint
learn.microsoft.com · source version/date: 2026-05

99. Microsoft Incident Response: Leveraging Windows Internals for Forensic Investigation
cdn-dynmedia-1.microsoft.com · source version/date: 2024

100. Microsoft Kernel-PnP event manifest (Windows 10 build 17134)
github.com · source version/date: 2020-03

101. Microsoft PowerShell event manifest (Windows 10 build 17134)
github.com · source version/date: 2020-03

102. Microsoft RdpCoreTS event manifest (Windows 10 build 17134)
github.com · source version/date: 2020-03

103. Microsoft WER-Diag event manifest (Windows 10 build 17134)
github.com · source version/date: 2020-03-28

104. Microsoft Windows 11 Security Book
microsoft.com · source version/date: 2023-09

105. Microsoft WMI-Activity event manifest (Windows 10 build 17134)
github.com · source version/date: 2020-03

106. Microsoft: Using the clipboard
support.microsoft.com · source version/date: 2026

References

107. Microsoft: Windows activity history and your privacy
support.microsoft.com · source version/date: 2026

108. MINIDUMP_HEADER (minidumpapiset.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

109. Monitor Delivery Optimization
learn.microsoft.com · source version/date: 2026-05

110. Navigating the Windows Mail database
eprints.whiterose.ac.uk · source version/date: 2018

111. Network Connection Status Indicator (NCSI) troubleshooting guidance - Windows Server
learn.microsoft.com · source version/date: 2026-06-30

112. NIST Special Publication (SP) 800-88 Rev. 1 (Withdrawn), Guidelines for Media Sanitization
csrc.nist.gov · source version/date: 2014-12-17

113. NVISO: Amcache contains SHA-1 Hash - It Depends!
blog.nviso.eu · source version/date: 2022-03

114. Offensive DPAPI With Nemesis
specterops.io · source version/date: 2026-03

115. OneDriveExplorer: SQLite database parser
github.com · source version/date: 2026-06-10

116. Per-user services
learn.microsoft.com · source version/date: 2024-10

117. PLOAD_IMAGE_NOTIFY_ROUTINE (ntddk.h) - Windows drivers
learn.microsoft.com · source version/date: 2026-05-29

118. PowerShell 7.4: script-block compilation and logging
github.com · source version/date: 2023

119. Profile loading fails - Windows Server
learn.microsoft.com · source version/date: 2026-06-30

120. PS_CREATE_NOTIFY_INFO (ntddk.h) - Windows drivers
learn.microsoft.com · source version/date: 2026-05-29

References

121. PsSetCreateThreadNotifyRoutineEx function (ntddk.h) - Windows drivers
learn.microsoft.com · source version/date: 2026-05-29

122. Recall overview - Windows apps
learn.microsoft.com · source version/date: 2026-06-25

123. Registry class - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

124. Registry Policy File Format
learn.microsoft.com · source version/date: 2018-05

125. RegQueryValueExW function (winreg.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

126. RegRipper: Internet Explorer TypedURLs parser
github.com · source version/date: 2026-05-27

127. RegRipper: NetworkList parser
github.com · source version/date: 2024-11

128. RegRipper: UserAssist parser
github.com · source version/date: 2021-05

129. Rekall: UserAssist binary layouts
github.com · source version/date: 2017-10

130. Reparse Points - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

131. Run and RunOnce Registry Keys - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

132. sc.exe config
learn.microsoft.com · source version/date: 2026-06-30

133. SDelete - Sysinternals
learn.microsoft.com · source version/date: 2026-06-19

134. Security guidelines for system services in Windows Server 2016
learn.microsoft.com · source version/date: 2026-06-30

References

135. Service host grouping in Windows 10
learn.microsoft.com · source version/date: 2017-07

136. Service Trigger Events - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

137. SetFileTime function (fileapi.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

138. SHAddToRecentDocs function (shlobj_core.h) - Win32 apps
learn.microsoft.com · source version/date: 2026-06-29

139. SmartScreen Policy CSP
learn.microsoft.com · source version/date: 2025-03

140. srumsvc.dll | System Resource Usage Monitor Service
strontic.github.io · source version/date: 2020

141. Start-Transcript (Microsoft.PowerShell.Host) - PowerShell
learn.microsoft.com · source version/date: 2026-06-30

142. Stopping a Service - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

143. Subscription (eventTriggerType) Element - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

144. Supporting Mount Manager Requests in a Storage Class Driver - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

145. Sygnia: Windows 11 Program Compatibility Assistant execution artifacts
sygnia.co · source version/date: version recorded in change log

146. Sysmon - Sysinternals
learn.microsoft.com · source version/date: 2026-06-19

147. SystemCollector
learn.microsoft.com · source version/date: 2017-05

148. Task Scheduler for developers - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

References

149. TaskScheduler library: TaskEvent event definitions
github.com · source version/date: 2020-02

150. TaskSettings.Hidden property - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

151. The system registry is no longer backed up to the RegBack folder starting in Windows 10 version 1803 - Windows Client
learn.microsoft.com · source version/date: 2026-06-30

152. TotalRecall: original Recall schema and security research (April 2026)
github.com · source version/date: 2026-04

153. Troubleshooting a Failover Cluster using Windows Error Reporting
learn.microsoft.com · source version/date: 2026-06-30

154. Troubleshooting a Failover Cluster using Windows Error Reporting
learn.microsoft.com · source version/date: 2026-06-30

155. Troubleshooting Firewall-Related Issues
learn.microsoft.com · source version/date: 2018-05

156. Update on Recall security and privacy architecture
blogs.windows.com · source version/date: 2024-09

157. Update: CIS Microsoft Windows 10 Enterprise (Release 1703) Benchmark v1.0.0
cisecurity.org · source version/date: 2017-11

158. Using Event Viewer with AppLocker
learn.microsoft.com · source version/date: 2024-09

159. Using the WebDAV Redirector
learn.microsoft.com · source version/date: 2014-03

160. Varieties of Kernel-Mode Dump Files - Windows drivers
learn.microsoft.com · source version/date: 2026-06-30

161. Velociraptor: Windows RecentApps artifact
github.com · source version/date: 2026-05

References

162. WER Settings - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

163. When did RecentApps go?
thinkdfir.com · source version/date: 2020-10

164. Windows 10 research: Sticky Notes SQLite queries
github.com · source version/date: 2023-12-19

165. Windows 7 Cool Feature - Problem Steps Recorder - Petri IT Knowledgebase
petri.com · source version/date: 2009-02

166. Windows 8 / Windows Server 2012: The New Swap File | Microsoft Community Hub
techcommunity.microsoft.com · source version/date: version recorded in change log

167. Windows Installer Cache
devblogs.microsoft.com · source version/date: 2005-11

168. Windows Installer Properties for the Uninstall Registry Key - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

169. Windows Push Notification Services (WNS) overview - Windows apps
learn.microsoft.com · source version/date: 2026-06-25

170. WindowsAI Policy CSP
learn.microsoft.com · source version/date: 2025

171. WindowsTimeline: original schema, activity types, and queries
github.com · source version/date: 2023-02

172. WMI Architecture - Win32 apps
learn.microsoft.com · source version/date: 2026-06-18

173. WMI-Activity Event 5858 ResultCode 0x80041032 - Windows Client
learn.microsoft.com · source version/date: 2026-06-30

References

174. Yogesh Khatri: Forensic implications of System Resource Usage Monitor (SRUM) data in Windows 8
sciencedirect.com · source version/date: 2015-03

175. YourPhoneAnalyzer: Windows Your Phone forensic parser
github.com · source version/date: 2021-01-06